



BITS Pilani
Pilani | Dubai | Goa | Hyderabad

BIRLA INSTITUTE OF TECHNOLOGY AND SCIENCE, PILANI
FIRST Semester 2021-2025

**“END-TO-END HYBRID INFRASTRUCTURE SETUP ON AZURE WITH RDS AND SCCM
INTEGRATION”**

CSIWZG628T DISSERTATION

by
Mansi Vashishth
2021wa86114

Dissertation Work carried out at

Wipro Technologies, PUNE

BIRLA INSTITUTE OF TECHNOLOGY AND SCIENCE
Pilani (Rajasthan) India

October **2025**

CSIWZG628T DISSERTATION

**“END-TO-END HYBRID INFRASTRUCTURE SETUP ON AZURE WITH RDS AND SCCM
INTEGRATION”**

Submitted in partial fulfilment of the requirements of
M. Tech Computing Systems & Infrastructure Degree Program

by
Mansi Vashishth
2021wa86114

Under the supervision of
Rishav Raj (Lead Administrator L2)

Dissertation work carried out at
Wipro Technologies, Pune

BIRLA INSTITUTE OF TECHNOLOGY AND SCIENCE
PILANI (RAJASTHAN)

October 2025

**BIRLA INSTITUTE OF TECHNOLOGY AND SCIENCE
PILANI**

Certificate

This is to clarify that the entitled Creating **End-to-End Hybrid Infrastructure Setup on Azure with RDS and SCCM Integration** submitted by Mansi Vashishth Bits-ID 2021wa86114 in partial fulfilment of the requirements of CSIWZG628T Dissertation, embodies the work done by her under my supervision.



Date: 31-October-2025

Signature of the supervisor

Rishav Raj

Designation Lead Administrator

ACKNOWLEDGEMENTS

I would like to thank my project Supervisor Rishav Raj for his guidance effort, guidance and patience. He has been extremely supportive and cooperative in this whole effort. His valuable comments and suggestions have been very helpful in enhancing the quality of work.

I would like to express my sincere gratitude towards my examiners Augustin Jabaraz and Preethi Manian for examining my progress despite their business schedule.

I am also thankful to WIPRO TECHNOLOGIES for giving me such opportunity for training in M. Tech Computing System and Infrastructure Degree Program and providing all the resources required for this dissertation.

BIRLA INSTITUTE OF TECHNOLOGY AND SCIENCE, PILANI

First Semester 2025- 26

**CSIW ZG628T DISSERTATION
FINAL EVALUATION FORM**

ID No. 2021WA86114 **Name of Student:** Mansi Vashishth

Name of Supervisor: Rishav Raj

Name of the Examiner(s): Augustin Jabaraz & Preethi Manian

Dissertation Title: "END-TO-END HYBRID INFRASTRUCTURE SETUP ON AZURE WITH RDS AND SCCM INTEGRATION"

Section II

(To be filled by the Supervisor in consultation with the examiner(s))

Comments on the dissertation work from Examiner and Supervisor (Select Y or N):

1. **Quantum of work**
 - a. **Justifiable as efforts for duration of work(16 weeks)** Y
 - b. **Work is in line with the commitments made in outline** Y
2. **Type of work**
 - a. **Client assignment** N
 - b. **Organization specific task** N
 - c. **General study project such as white paper** N
 - d. **Any other (kindly elaborate below in a line or two if Y)** Y
Deploying Windows domain in Azure environment via Automation
3. **Nature of work**
 - a. **Routine in nature** Y
 - b. **Involved creativity and rational thinking** Y

Kindly elaborate below if answer for above is "Y"

It's a detailed study on Windows and its components especially the SQL, SCCM and RDS farm using terraform and powershell to automate the infra and setup. The Knowledge gained by Mansi with this project is very helpful to maintain SCCM environment on client side.

4. Evaluation methodology

- a. Evaluation done based on presentation to supervisor and examiner Y
- b. Evaluation done through Viva conducted by supervisor and examiner Y
- c. Student regularly interacted with supervisor and incorporated the suggestions made Y
- d. Brief description on the report submitted and quality of presentation

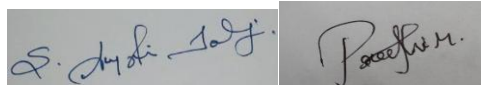
It's a detailed study on Windows and its components especially the SQL, SCCM and RDS farm using terraform and powershell to automate the infra and setup. The Knowledge gained by Mansi with this project is very helpful to maintain SCCM environment on client side. She have done **excellent** job in explaining the task.

5. Final evaluation matrix

Tick the appropriate box (1 is lowest and 5 is the highest)

Dimension	Rankà	1	2	3	4	5
Student abilities in general						
Understanding of the subject of dissertation						5
Creative thinking ability to come up with new ideas						5
						5
Viva / Seminar presentation						
Communication ability						5
Organization of material						5
Response to review questions						5
Cohesive thinking ability						5
Report submitted						
Report structure and format						5
Technical content of the report						5
Explanation on the significance of the assignment						5
Analysis of alternative approaches						5
Recommendation and suggestions						5
Discussion on benefits and limitations						5

Any other comments: Excellent performance



Signature of examiner(s)



Signature of Supervisor

Date: 31-Oct-2025

S.No	Content	Page No.
1.	Abstract	8
2.	Objective and scope of the plan	9
3.	Create and Azure Subscription	10
4.	Resource group, Virtual Network and Virtual Machine	11- 12
5.	Tools used in this project	12
6.	Steps and Process to create VM setup using Terraform	13-15
7.	Verify the VM created	16
8.	Connect VM via RDP	17-18
9.	Install ADDS and promote the server as Domain Controller	19-21
10.	Create a DNS server	22
11.	Create OU & Users	23
12.	SQL server Setup	24-26
13.	Script to perform SQL and SSMS setup	27
14.	Terraform Script to Create 3 Virtual Machines	28-34
15.	Windows Server Update Services (WSUS)	35
16.	What is SCCM / MECM?	36
17.	SCCM setup installation	37-44
18.	SCCM Configuration After Installation	45-50
19.	RDS Farm	51
20.	Components of RDS farm	52-53
21.	Step by step commands performed to setup RDS Farm	54-55
22.	Creating Authorization policy for RD Gateway	56-58
23.	Testing RDP connection through RD Gateway	59-62
24.	Conclusion	63
25.	Detailed Plan of the work	64
26.	Reference Links	65
27.	Check list of items for report	66

Abstract

In this project we demonstrate the deployment of 3 windows server infrastructure in Microsoft Azure with the help of tools like Terraform and PowerShell for automate the process. This infrastructure included three Windows Server VMs, Resource group, virtual network, subnet, public Ips all these were provisioned through Terraform.

After deployment, the first VM was promoted to a Domain Controller with roles such as AD DS and DNS. The remaining VMs were joined to the domain and configured with their respective roles using automated PowerShell scripts. The second VM was set up with SQL Server, while the third VM was added as a domain member server for SCCM. SCCM was installed and configured on mansivm-1 to centrally manage client updates. Remote Desktop Services (RDS) roles were deployed on mansivm-2, with mansivm-0 and mansivm-1 functioning as Session Hosts. PowerShell scripts were used to automate the entire RDS setup, including role installation, certificate binding, session collection creation, load balancing, and RD Gateway configuration.

This project demonstrates the approach in which Terraform provisions infrastructure and automated server role configuration using PowerShell that enables a scalable and production ready setup. The use of Azure ensures flexibility, security, and cost-effective scalability across the environment.

Objective and scope of the plan

Objective:

Main Objective is to bring all the infrastructure of an existing organization under a single domain using windows technologies based on cloud computing using Terraform and PowerShell maintain the infrastructure in compliance all the time.

Scope of the plan:

The scope of the plan is to create a windows domain with various windows technologies that are required for an organization using Cloud Computing.

- Get an Azure subscription
- The automation and deployment of three virtual machines on Azure using Terraform.
- Deploy the Windows Machines
- Create Domain
- Promote the VM0 to domain controller.
- Deploy all the applications/ roles using PowerShell.

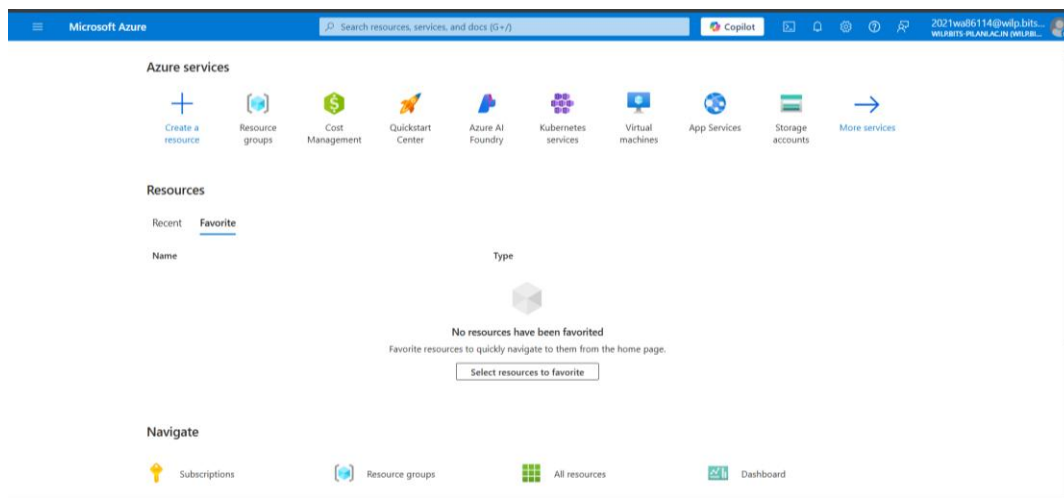
This project can be used to train my team on Azure and automation which can be show cased to the client and get the contract of cloud-based projects that are the handled by client.

Create an Azure Account

Creating an Azure account is as easy as you create outlook account. Organizations will buy subscriptions to maintain the resources and secure the environment.

Here in this project, I have used Azure free subscription also called as Student trail.
Create an Outlook account, and then use the outlook account to create azure free subscriptions by signing up at <https://portal.azure.com/#home>

Below is the home screen of Azure after sign-up

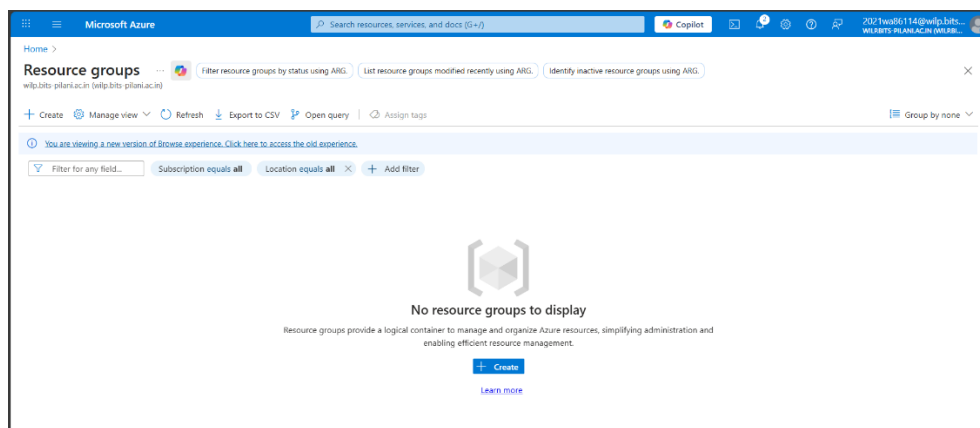


Resource group:

A **Resource Group** in Azure is like a folder where you keep all your Azure resources together. It helps you organize and manage everything in one place — for example, virtual machines, networks, or storage accounts.

Even though the resource group itself is created in a specific region, it can still hold resources from other regions. There's no limitation on that.

You can think of it as the **main container** where all your Azure resources live and are managed
In my case name of resource is **MTECH**.



Virtual Network:

Azure (VNet) is the core blocks for creating private networks inside Azure. It allows different resources—like (VMs)—to talk to each other securely, as well as connect to the internet or you're on-premises environment.

You can think of a VNet as similar to the networks we use in traditional data centers, but it also comes with Azure's advantages like easy scaling, high availability, and strong isolation.

By default, everything inside a VNet can connect out to the internet. If you want the internet or outside users to reach a resource inside the VNet, you need to give it a public IP address or put it behind a load balancer. You can also manage outgoing connections using public IPs or load balancers.

I have named the VNet as **MTech_Network**.

Virtual Machine:

Azure Virtual Machines (VMs) are a type of cloud-based computer that you can create and use whenever you need. Unlike other ready-made Azure services, VMs give you more freedom and control over the system because you decide how it is set up, which software to install, and how it is maintained.

The main advantage of using a VM in Azure is that you don't need to buy or manage the physical servers yourself. Azure takes care of the underlying hardware, and you just focus on using the machine. However, you are still responsible for things like updating the operating system, installing applications, and keeping it secure. There are different situations where Azure VMs are useful:

- **Development and Testing** – Developers can quickly spin up VMs with the exact settings they need to build and test their applications.
- **Running Applications** – If your app has changing demand, you can add more VMs when traffic is high and remove them when it's low. This way, you only pay for what you actually use.
- **Extending Your Datacenter** – VMs in Azure can connect with your company's on-premises network, making it easy to expand your IT infrastructure without buying new physical servers.

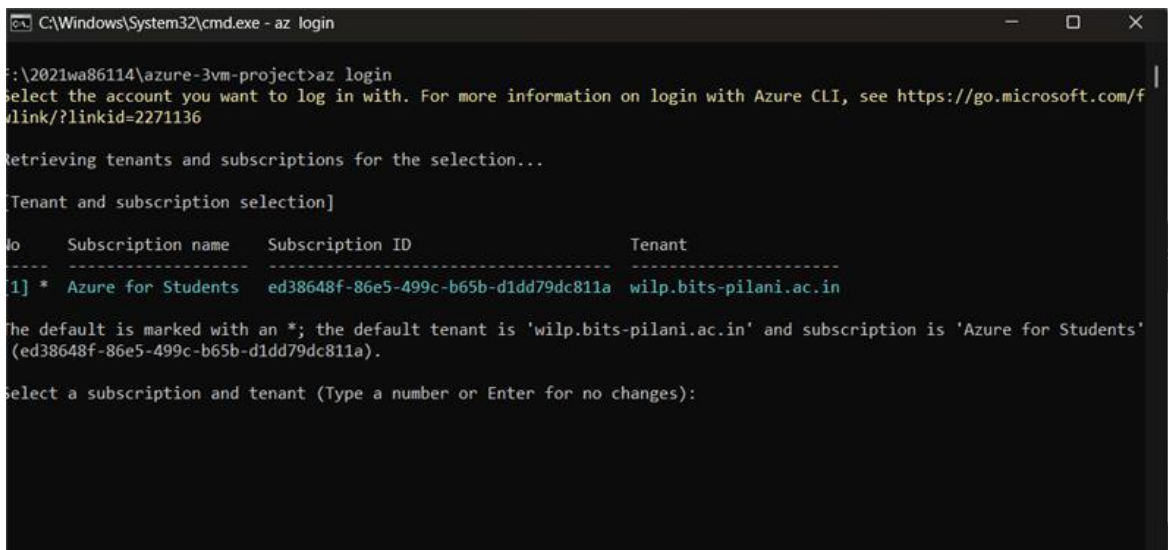
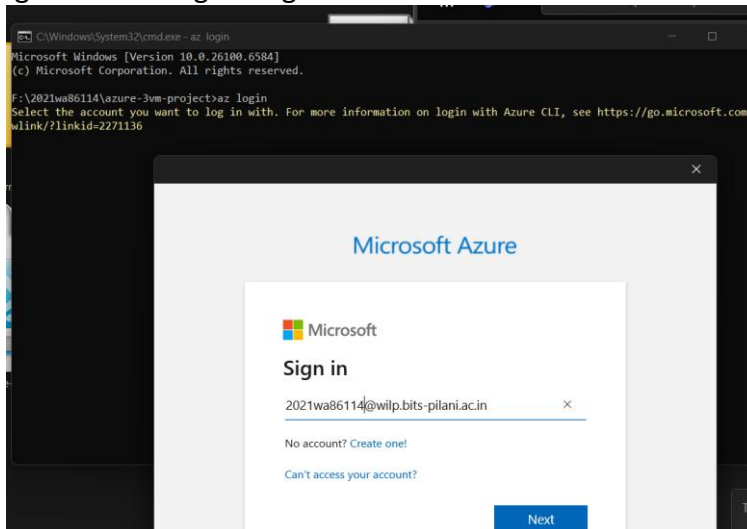
Here, I have created 3 VM's named as: mansivm-0, mansivm-1, mansivm-2.

Tools Used in This Project

1. **Terraform** – This is a tool that helps us create servers and networks in Azure automatically, instead of doing everything by hand.
2. **Microsoft Azure Portal** – The website where we can see, manage, and monitor all the resources we create in Azure.
3. **Azure Virtual Machines (VMs)** – These are like computers running in the cloud. We used them for Domain Controller and SQL Server.
4. **PowerShell** – A scripting tool we used to run commands for setting up the Domain Controller, joining other VMs to the domain, and installing SQL.
5. **Remote Desktop (RDP)** – This allows us to log in to our virtual machines and work on them as if they were normal computers.

Steps and Process to Create the setup now using Terraform

1. Created Infrastructure with Terraform – Used Terraform to automatically create 3 virtual machines inside an Azure Virtual Network:
 - Pework: a. Install and setup Terraform in local machine.
 - b. Download Azure CLI.
 - c. If using VS Code, then install extension – HashiCorp Terraform for smooth setup.
 - Created a folder name: F:\2021wa86114\azure-3vm-project to store script and terraform scripts.
 - Open Command Prompt as admin and navigate to the folder path where you have stored the scripts and .tf code.
 - Login Azure using: az login



***I have given below names in configuration setup:

- Resource group name: MTECH
- Azure region for resources: Central India
- VNet Name: MTech_Network
- Subnet Name: subnet-project
- Number of Virtual Machines to create: 3
- Domain name: bits.com

***** I have documented the terraform code for this process on page no 28

- Initialize Terraform: Run **terraform init**

First, I ran terraform init to set up Terraform and download the required Azure provider.

```
F:\2021wa86114\azure-3vm-project>terraform init
Initializing the backend...
Initializing provider plugins...
- Reusing previous version of hashicorp/azurerm from the dependency lock file
- Using previously-installed hashicorp/azurerm v3.117.1

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.

F:\2021wa86114\azure-3vm-project>
```

- Terraform Plan: Run **terraform plan**

Then, I used terraform plan to check what resources Terraform would create in Azure. This step helps to make sure everything is correct before actually creating the resources.

```
C:\Windows\System32\cmd.exe
F:\2021wa86114\azure-3vm-project>terraform plan

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the
following symbols:
+ create

Terraform will perform the following actions:

# azurerm_network_interface.nic[0] will be created
+ resource "azurerm_network_interface" "nic" {
+   accelerated_networking_enabled = (known after apply)
+   applied_dns_servers             = (known after apply)
+   dns_servers                    = (known after apply)
+   enable_accelerated_networking = (known after apply)
+   enable_ip_forwarding           = (known after apply)
+   id                             = (known after apply)
+   internal_domain_name_suffix    = (known after apply)
+   ip_forwarding_enabled          = (known after apply)
+   location                       = "centralindia"
+   mac_address                    = (known after apply)
+   name                           = "nic-0"
+   private_ip_address             = (known after apply)
+   private_ip_addresses           = (known after apply)
+   resource_group_name            = "MTECH"
+   virtual_machine_id             = (known after apply)

+   ip_configuration {
+     gateway_load_balancer_frontend_ip_configuration_id = (known after apply)
+     name                                               = "ipconfig-0"
+     primary                                           = (known after apply)
  }
}
```

- Terraform Apply: **terraform apply -auto-approve**

Next, I ran terraform apply to actually create the resources. Terraform created a resource group, virtual network, subnet, network security groups, public IPs, network interfaces, and three virtual machines.

```
C:\Windows\System32\cmd.exe

Note: You didn't use the -out option to save this plan, so Terraform can't guarantee to take exactly these actions if you run "terraform apply" now.

F:\2021wa86114\azure-3vm-project>

F:\2021wa86114\azure-3vm-project>terraform apply -auto-approve

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
+ create

Terraform will perform the following actions:

# azurerm_network_interface.nic[0] will be created
+ resource "azurerm_network_interface" "nic" {
  + accelerated_networking_enabled = (known after apply)
  + applied_dns_servers            = (known after apply)
  + dns_servers                   = (known after apply)
  + enable_accelerated_networking = (known after apply)
  + enable_ip_forwarding          = (known after apply)
  + id                            = (known after apply)
  + internal_domain_name_suffix   = (known after apply)
  + ip_forwarding_enabled         = (known after apply)
  + location                      = "centralindia"
  + mac_address                   = (known after apply)
  + name                          = "nic-0"
  + private_ip_address            = (known after apply)
  + private_ip_addresses          = (known after apply)
}
```

```
C:\Windows\System32\cmd.exe

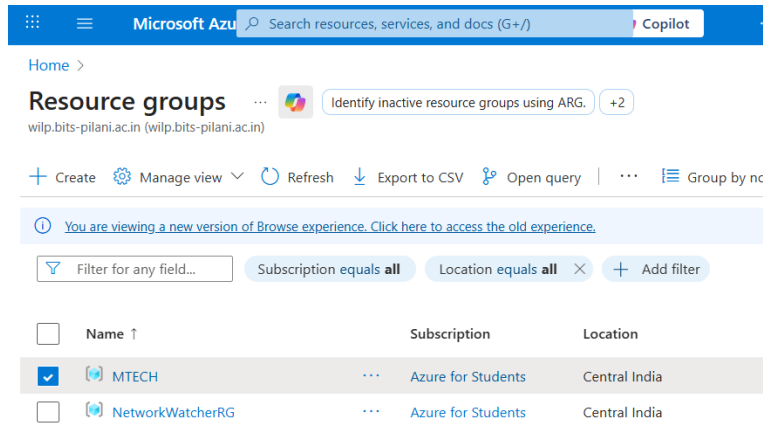
Outputs:

domain_controller_ip = "10.0.1.6"
print_server_ip      = "10.0.1.4"
resource_group_location = "centralindia"
resource_group_name   = "MTECH"
sql_server_ip         = "10.0.1.5"
subnet_id             = "/subscriptions/ed38648f-86e5-499c-b65b-d1dd79dc811a/resourceGroups/MTECH/providers/Microsoft.Network/virtualNetworks/MTEch_Network/subnets/subnet-project"
virtual_network_id     = "/subscriptions/ed38648f-86e5-499c-b65b-d1dd79dc811a/resourceGroups/MTECH/providers/Microsoft.Network/virtualNetworks/MTEch_Network"
virtual_network_name   = "MTEch_Network"
vm_names = [
  "mansivm-0",
  "mansivm-1",
  "mansivm-2",
]
vm_private_ips = [
  "10.0.1.6",
  "10.0.1.5",
  "10.0.1.4",
]
vm_public_ips = [
  "4.213.166.86",
  "4.213.166.182",
  "4.213.181.129",
]

F:\2021wa86114\azure-3vm-project>
```

Post terraform apply: Verify Resources

After running the Terraform script, I went to the Azure portal to confirm that all three virtual machines and the related network resources were created correctly.



Microsoft Azure Search resources, services, and docs (G+ /) Copilot

Home > Resource groups

Identify inactive resource groups using ARG. +2

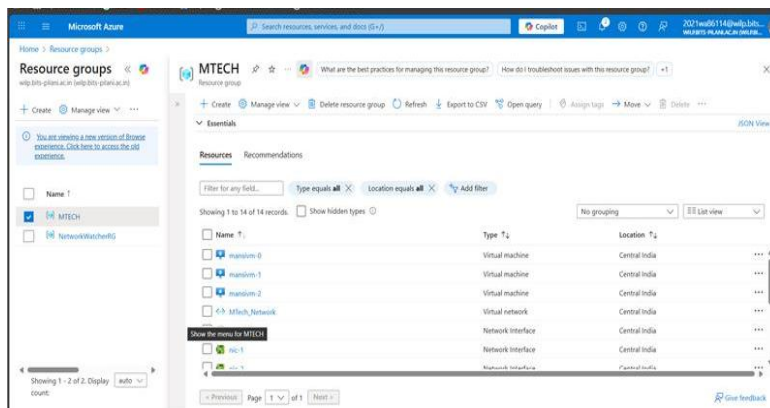
wilp.bits-pilani.ac.in (wilp.bits-pilani.ac.in)

+ Create Manage view Refresh Export to CSV Open query Group by nc

You are viewing a new version of Browse experience. Click here to access the old experience.

Filter for any field... Subscription equals all Location equals all Add filter

Name	Subscription	Location
MTECH	Azure for Students	Central India
NetworkWatcherRG	Azure for Students	Central India



Microsoft Azure Search resources, services, and docs (G+ /) Copilot

Home > Resource groups > MTECH

What are the best practices for managing this resource group? How do I troubleshoot issues with this resource group? +1

+ Create Manage view Delete resource group Refresh Export to CSV Open query Assign tags Move Delete

Essentials JSON View

Resources Recommendations

Filter for any field... Type equals all Location equals all Add filter

Showing 1 to 14 of 14 records Show hidden types No grouping List view

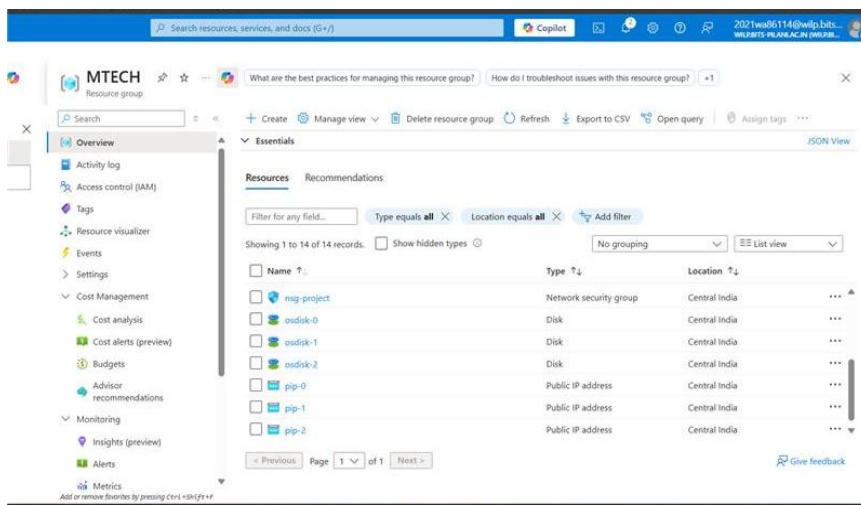
Name	Type	Location
vm1	Virtual machine	Central India
vm2	Virtual machine	Central India
vm3	Virtual machine	Central India
nic1	Network interface	Central India
nic2	Network interface	Central India

Show the menu for MTECH

Showing 1 - 2 of 2. Display counts

Previous Page 1 of 1 Next

Give feedback



Microsoft Azure Search resources, services, and docs (G+ /) Copilot

Home > Resource groups > MTECH

What are the best practices for managing this resource group? How do I troubleshoot issues with this resource group? +1

+ Create Manage view Delete resource group Refresh Export to CSV Open query Assign tags

Overview Activity log Access control (IAM) Tags Resource visualizer Events Settings Cost Management Cost analysis Cost alerts (preview) Budgets Advisor recommendations Monitoring Insights (preview) Alerts Metrics

Essentials JSON View

Resources Recommendations

Filter for any field... Type equals all Location equals all Add filter

Showing 1 to 14 of 14 records Show hidden types No grouping List view

Name	Type	Location
nsg-project	Network security group	Central India
osdisk-0	Disk	Central India
osdisk-1	Disk	Central India
osdisk-2	Disk	Central India
pip-0	Public IP address	Central India
pip-1	Public IP address	Central India
pip-2	Public IP address	Central India

Previous Page 1 of 1 Next

Give feedback

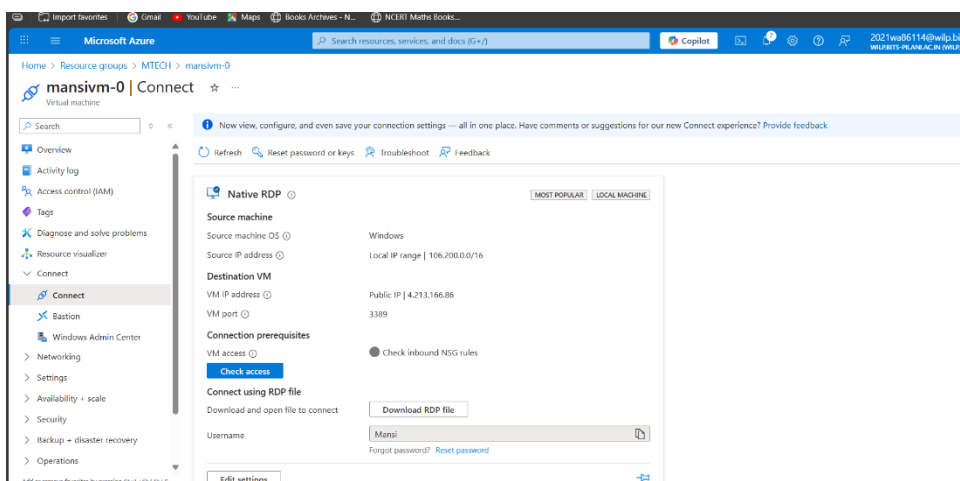
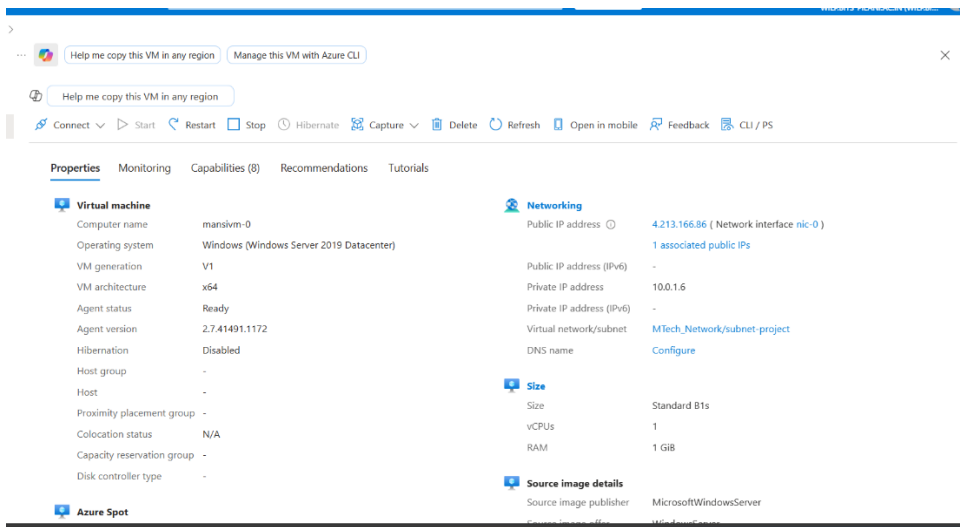
Add or remove favorites by pressing ctrl + v (F1 + F)

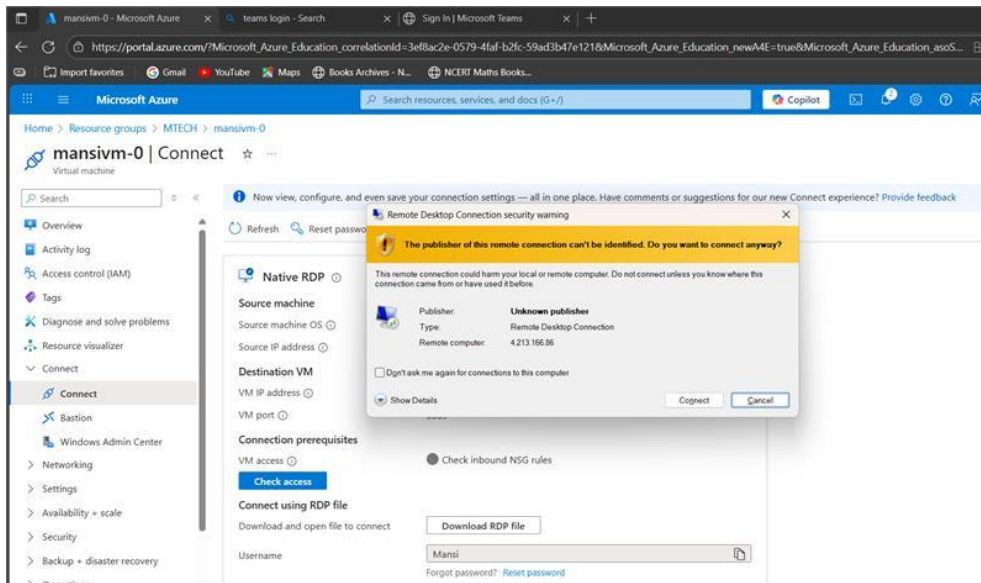
Promote Mansivm-0 to act as Domain Controller

Each operating system uses its own way (protocol) to connect to a virtual machine. In this project, the VMs are running Windows Server 2019 Datacenter. Windows servers use Remote Desktop Protocol (RDP), which works on port 3389, to connect.

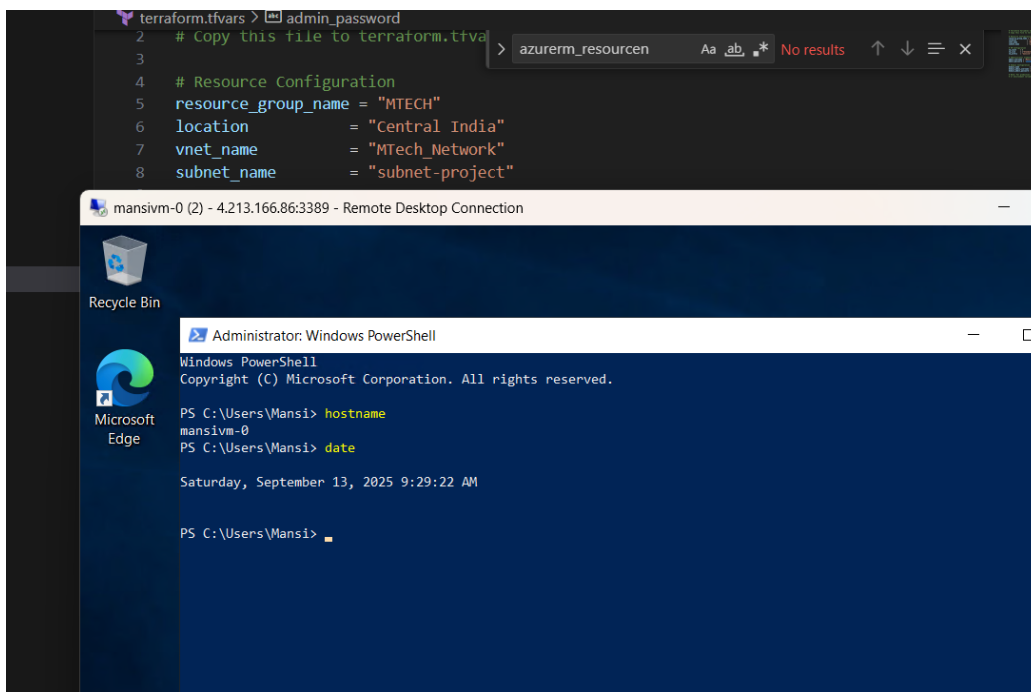
To log in, we can simply download the RDP file from the portal and connect by entering the admin username and password that were set while creating the VM.

- ❖ Step 1: Connect Virtual machine – mansivm-0 via rdp
Click on server name -> connect -> Download RDP file - connect





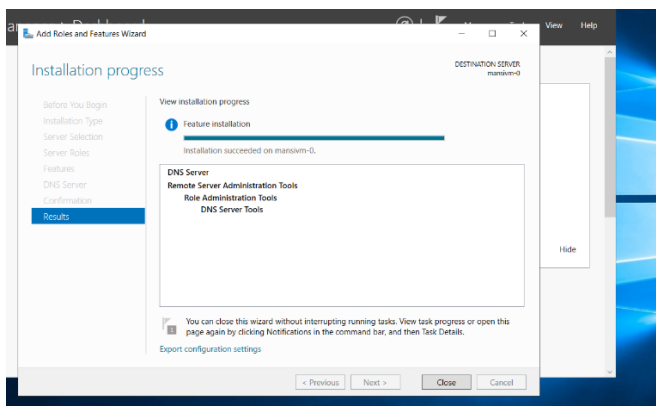
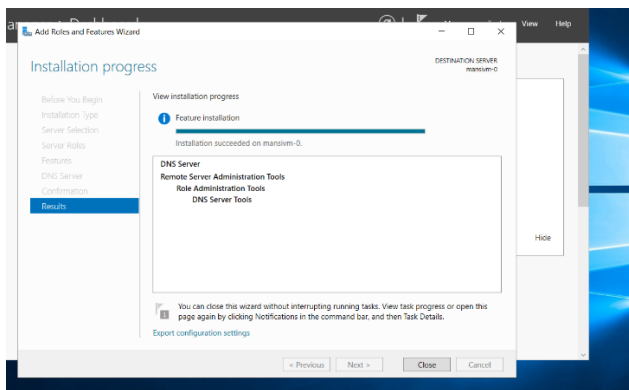
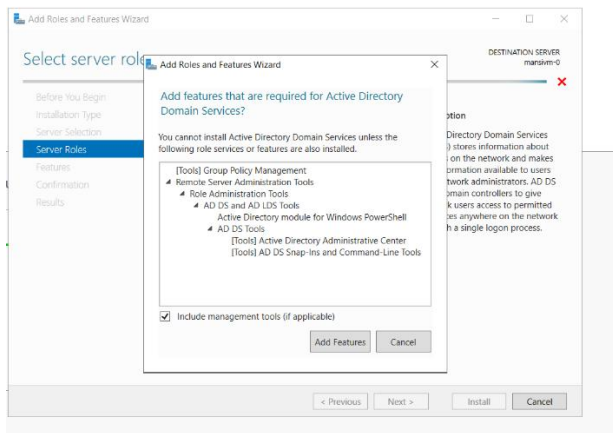
I am able to get into the server successfully. Since its free trail account the firewalls are already configured by Azure and Microsoft Defender is the antivirus tool installed in the server.



Install ADDS and promote the server as Domain Controller

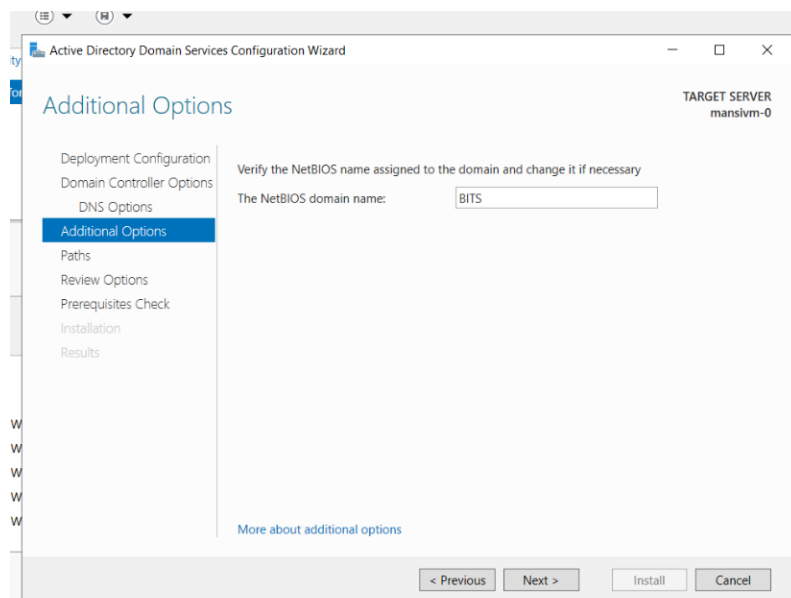
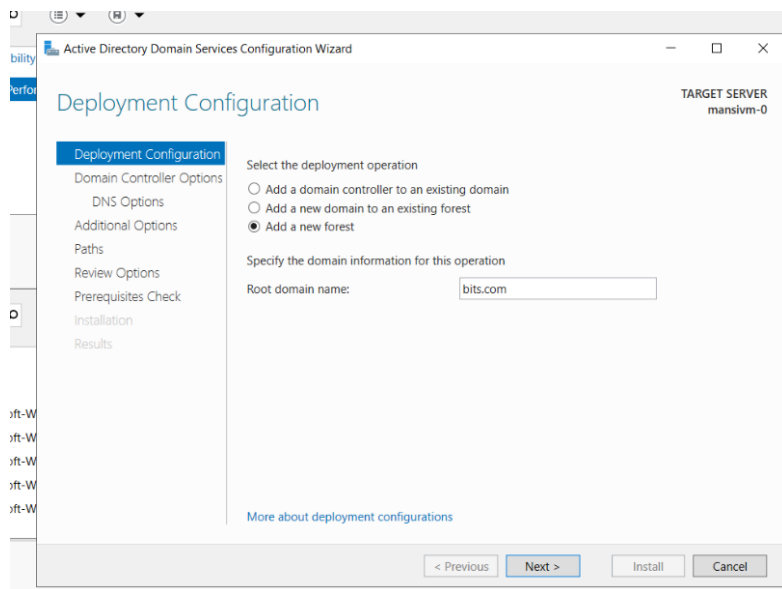
A Domain Controller (DC) is a special server in a network that manages security and user access. It checks login requests, stores user accounts, and makes sure security rules are followed. In Windows environments, the Domain Controller works with Active Directory, which is the main service used to organize and control users and resources in a domain.

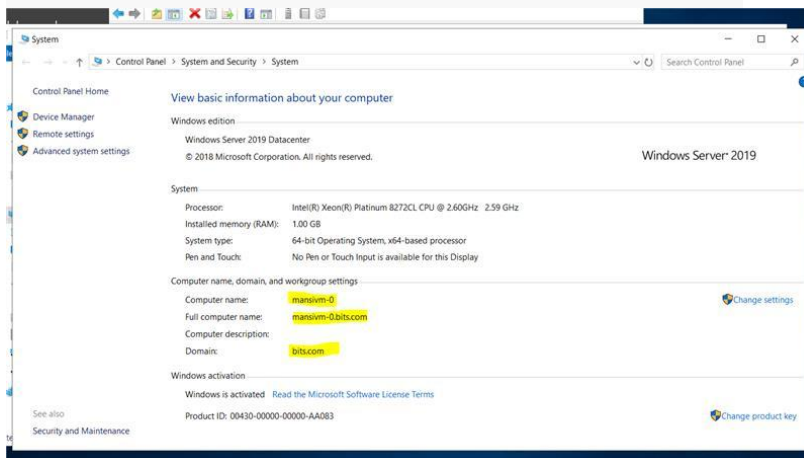
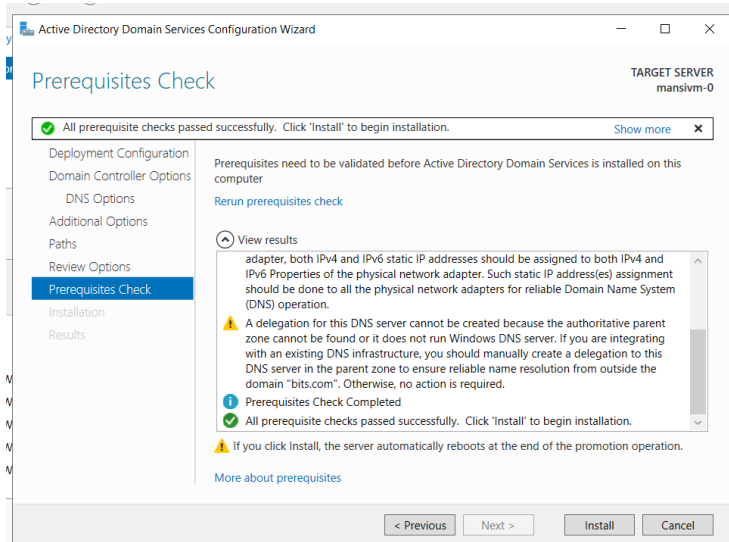
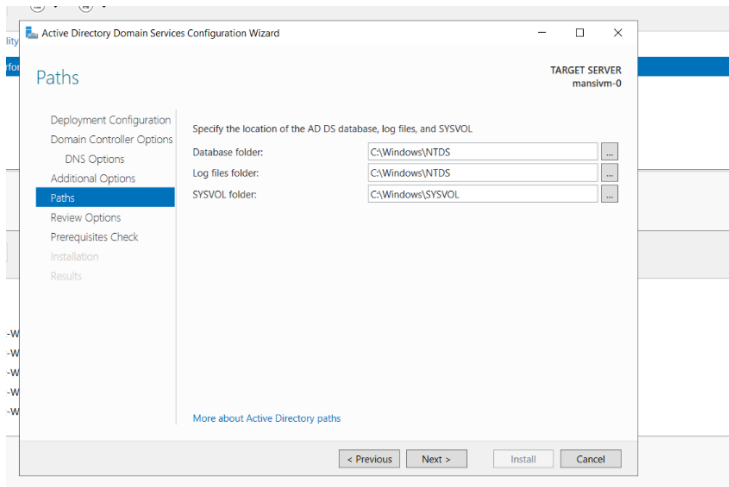
Open Server Manager → Add Roles and Features → Jump to the server roles → Select Active Directory Domain Services role → Add all the features → Click Next → Install



Post installation of the AD DS role, the server can be promoted to a domain controller. Since I was creating a new domain, I chose the option “Add a new forest” and named my domain bits.com

Next, I set a password for Active Directory recovery and continued with the default settings. Once the installation finished, the server rebooted, and after restart, a new domain called bits.com was successfully created, managed by the forest root domain controller (DC).





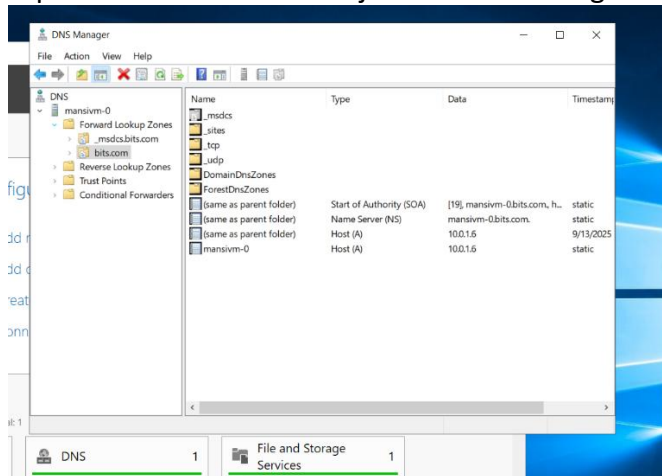
Create a DNS server:

The Domain Name System (DNS) is like a naming service for computers and servers. It makes it possible to use simple names instead of remembering long numbers (IP addresses). For example, instead of typing an IP, we just use a name like server1 or google.com.

When we install DNS, the server can match names to the right IP addresses. This way, users can easily find and connect to other systems without needing to know the actual numbers.

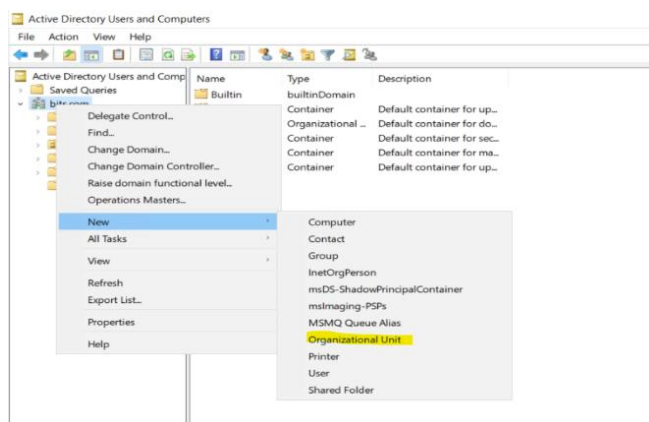
The process of installing DNS is almost the same as installing ADDS, but here we just select DNS. After the server restarts, the same domain controller also works as a DNS server.

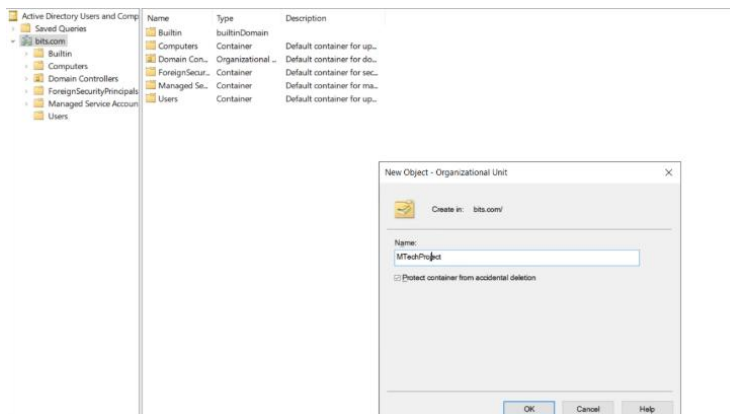
In small setups, one server can handle both ADDS and DNS. But in bigger companies, there are usually separate servers dedicated just for DNS management.



An OU is like a folder inside Active Directory where you can keep users, groups, and computers of an organization. It helps in keeping everything organized.

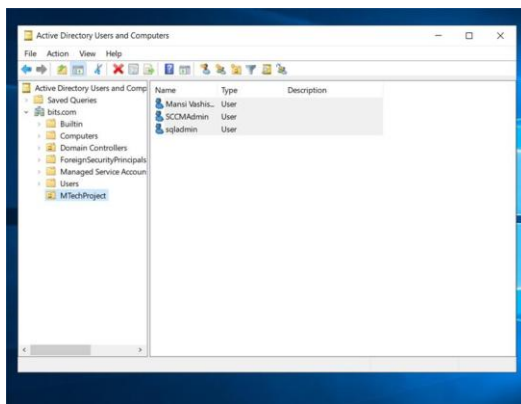
The main use of an OU is that administrators can apply policies (rules) or permissions to it. For example, you can create an OU for each department like HR, IT, or Finance, and then apply specific rules only to that department.



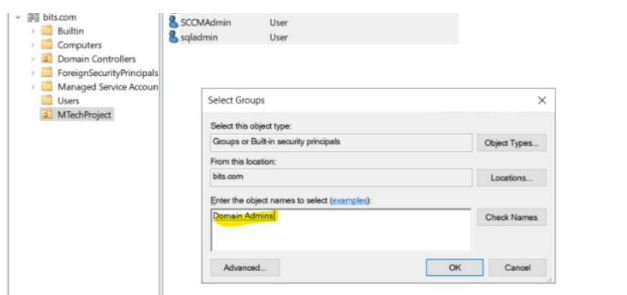


Once the OU is ready, create users to manage the environment, services and applications. Below are the users that I have created in this OU.

- **Mansi Vashishth**
- **SCCMAdmin**
- **sqladmin**



I have added all the users to **Domain Admins** so that they will have admin access over the Domain.



As server is successfully promoted as domain controller. Now I have joined the rest two vms to domain -bits.com and configure the roles as planned.

Provided Admin access to the servers for the accounts that which we created.

- **Mansi Vashishth** has full control over Domain Controller (DC).
- **SCCMAdmin** has the full control over **SCCM** server.
- **Sqladmin** has full control over **SQL** server.

Configure SQL Server setup on mansivm-1

- Login to server via rdp.
- Now, to join server to domain bits.com we need to configure VM 0 ip as DNS server in rest of the servers by using cli.

\$DomainName = "bits"

\$DC_IP = "10.0.1.6"

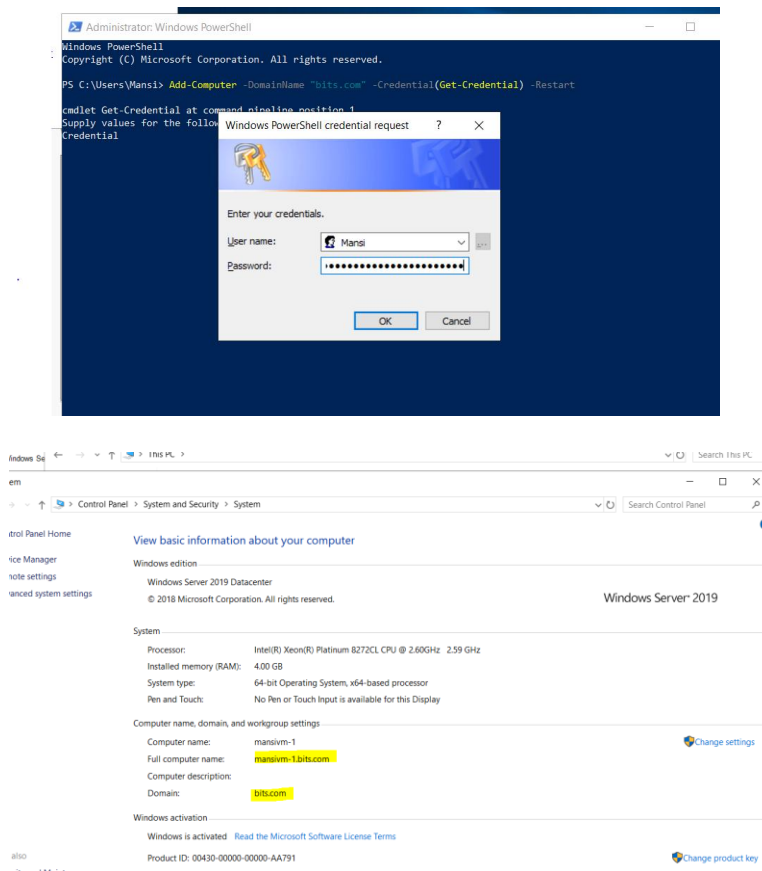
Set DNS to point to the domain controller

Set-DnsClientServerAddress -InterfaceAlias "Ethernet" -ServerAddresses \$DC_IP

- Once, DNS configured successfully. Run below command on vm to join to the domain.

Join domain

Add-Computer -DomainName "bits.com" -Credential(Get-Credential) -Restart



SQL Server 2019

SQL Server 2019 is Microsoft's database software. It is used to store, manage, and organize data so that applications can use it. For example, if you have an application that keeps records of students, employees, or sales, SQL Server is the place where all that data is saved.

It is reliable, secure, and can handle large amounts of information. SQL Server 2019 also comes with many features to make searching, analysing, and protecting data easier.

SQL Server Management Studio (SSMS)

This is a free that is provided by Microsoft to work with SQL Server. It gives you simple window where you can:

- Connect to your SQL Server,
- Create or manage databases,
- Run queries (ask questions from the database),
- Backup or restore data, and
- Monitor the server.

In this project, I installed SQL Server 2019 Express

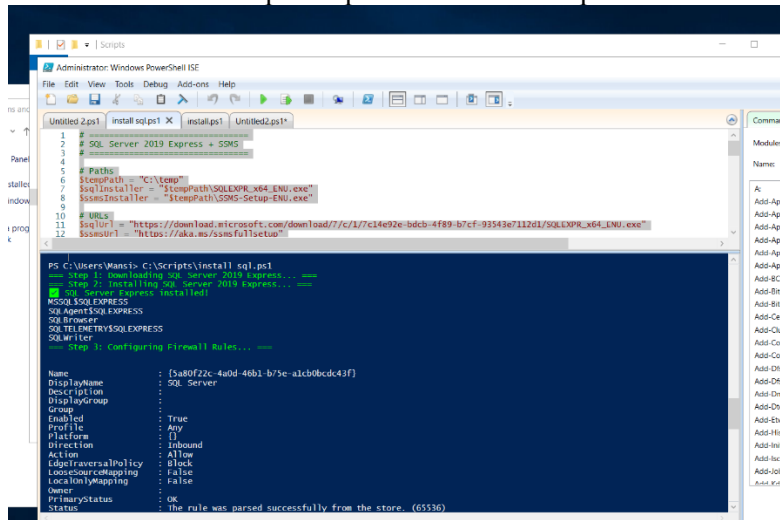
Instead of doing the setup manually, I used a PowerShell script to make the process faster and easier.

The script performed the following steps automatically:

1. Downloaded the SQL Server 2019 Express installer.
2. Installed SQL Server with the required configuration.
3. Opened the firewall port (1433) so that SQL Server can be accessed.
4. Downloaded and installed SQL Server Management Studio (SSMS).
5. Cleaned up the temporary setup files after installation.

By using the script, the installation became more reliable and repeatable. I didn't need to click through the installer manually; everything was done in one go.

I have created the script and paster under C:\Scripts of VM1



***** Script to perform SQL and SSMS setup installation**

```
# Paths
$tempPath = "C:\temp"
$sqlInstaller = "$tempPath\SQLEXPR_x64_ENU.exe"
$ssmsInstaller = "$tempPath\SSMS-Setup-ENU.exe"

# URLs
$sqlUrl = "https://download.microsoft.com/download/7/c/1/7c14e92e-bdcb-4f89-b7cf-93543e7112d1/SQLEXPR_x64_ENU.exe"
$ssmsUrl = "https://aka.ms/ssmsfullsetup"

# Create temp folder
if (!(Test-Path $tempPath)) { New-Item -ItemType Directory -Path $tempPath -Force }
Write-Host "=== Step 1: Downloading SQL Server 2019 Express... ===" -ForegroundColor Green
Invoke-WebRequest -Uri $sqlUrl -OutFile $sqlInstaller -UseBasicParsing
Write-Host "=== Step 2: Installing SQL Server 2019 Express... ===" -ForegroundColor Green
$args = @(
    "/Q",
    "/ACTION=Install",
    "/FEATURES=SQLEngine",
    "/INSTANCENAME=SQLEXPRESS",
    "/SQLSVCACCOUNT=`"NT AUTHORITY\SYSTEM`"",
    "/SQLSYSADMINACCOUNTS=`"BITS\sqladmin`"",
    "/TCPENABLED=1",
    "/IACCEPTSQLSERVERLICENSETERMS"
)
Start-Process -FilePath $sqlInstaller -ArgumentList $args -Wait -NoNewWindow
Write-Host "SQL Server Express installed!" -ForegroundColor Green

# Check SQL services
Get-Service *SQL* | Write-Host
Write-Host "=== Step 3: Configuring Firewall Rules... ===" -ForegroundColor Green
New-NetFirewallRule -DisplayName "SQL Server" -Direction Inbound -Protocol TCP -LocalPort 1433 -Action Allow -ErrorAction SilentlyContinue
New-NetFirewallRule -DisplayName "SQL Browser" -Direction Inbound -Protocol UDP -LocalPort 1434 -Action Allow -ErrorAction SilentlyContinue
Write-Host "Firewall configured!" -ForegroundColor Green

Write-Host "=== Step 4: Downloading SSMS... ===" -ForegroundColor Green
Invoke-WebRequest -Uri $ssmsUrl -OutFile $ssmsInstaller -UseBasicParsing
Write-Host "=== Step 5: Installing SSMS... (this may take a while) ===" -ForegroundColor Green
```

******* Terraform Script to Create 3 Virtual Machines**

Below is the Terraform script that was used in this project to create:

- A resource group
- A virtual network and subnet
- A network security group with rules
- Public IPs and Network Interfaces
- Three Virtual Machines (VM0 as Domain Controller, VM1 as SQL Server, and VM2 as RDS)

Main.tf:

```
// -----  
// Provider Configuration  
// -----  
terraform {  
  required_version = ">= 1.0"  
  required_providers {  
    azurerm = {  
      source = "hashicorp/azurerm"  
      version = "~> 3.0"  
    }  
  }  
}  
  
provider "azurerm" {  
  features {  
    resource_group {  
      prevent_deletion_if_contains_resources = false  
    }  
  }  
}  
  
// -----  
// Resource Group  
// -----  
resource "azurerm_resource_group" "rg" {  
  name      = var.resource_group_name  
  location = var.location  
}  
  
// -----  
// Virtual Network  
// -----  
resource "azurerm_virtual_network" "vnet" {  
  name            = var.vnet_name  
  address_space   = ["10.0.0.0/16"]  
}
```



```

location          = azurerm_resource_group.rg.location
resource_group_name = azurerm_resource_group.rg.name
}
resource "azurerm_subnet" "subnet" {
  name                = var.subnet_name
  resource_group_name = azurerm_resource_group.rg.name
  virtual_network_name = azurerm_virtual_network.vnet.name
  address_prefixes     = ["10.0.1.0/24"]
}
// -----
// Network Security Group
// -----
resource "azurerm_network_security_group" "nsg" {
  name                = "nsg-project"
  location            = azurerm_resource_group.rg.location
  resource_group_name = azurerm_resource_group.rg.name
}
// NSG Rules for RDP access
resource "azurerm_network_security_rule" "rdp" {
  name                = "RDP"
  priority            = 1000
  direction           = "Inbound"
  access              = "Allow"
  protocol            = "TCP"
  source_port_range   = "*"
  destination_port_range = "3389"
  source_address_prefix = "*"
  destination_address_prefix = "*"
  resource_group_name   = azurerm_resource_group.rg.name
  network_security_group_name = azurerm_network_security_group.nsg.name
}
// NSG Rules for Active Directory
resource "azurerm_network_security_rule" "ad_dns" {
  name                = "AD-DNS"
  priority            = 1001
  direction           = "Inbound"
  access              = "Allow"
  protocol            = "TCP"
  source_port_range   = "*"
  destination_port_range = "53"
  source_address_prefix = "10.0.1.0/24"
  destination_address_prefix = "*"
  resource_group_name   = azurerm_resource_group.rg.name
  network_security_group_name = azurerm_network_security_group.nsg.name
}

```

```

resource "azurerm_network_security_rule" "ad_ldap" {
  name                = "AD-LDAP"
  priority             = 1002
  direction           = "Inbound"
  access              = "Allow"
  protocol             = "Tcp"
  source_port_range    = "*"
  destination_port_range = "389"
  source_address_prefix = "10.0.1.0/24"
  destination_address_prefix = "*"
  resource_group_name   = azurerm_resource_group.rg.name
  network_security_group_name = azurerm_network_security_group.nsg.name
}

resource "azurerm_network_security_rule" "ad_ldaps" {
  name                = "AD-LDAPS"
  priority             = 1003
  direction           = "Inbound"
  access              = "Allow"
  protocol             = "Tcp"
  source_port_range    = "*"
  destination_port_range = "636"
  source_address_prefix = "10.0.1.0/24"
  destination_address_prefix = "*"
  resource_group_name   = azurerm_resource_group.rg.name
  network_security_group_name = azurerm_network_security_group.nsg.name
}

resource "azurerm_network_security_rule" "ad_kerberos" {
  name                = "AD-Kerberos"
  priority             = 1004
  direction           = "Inbound"
  access              = "Allow"
  protocol             = "Tcp"
  source_port_range    = "*"
  destination_port_range = "88"
  source_address_prefix = "10.0.1.0/24"
  destination_address_prefix = "*"
  resource_group_name   = azurerm_resource_group.rg.name
  network_security_group_name = azurerm_network_security_group.nsg.name
}

// NSG Rules for SQL Server
resource "azurerm_network_security_rule" "sql_server" {
  name                = "SQL-Server"
  priority             = 1005
  direction           = "Inbound"

```

```

access                = "Allow"
protocol              = "Tcp"
source_port_range     = "*"
destination_port_range = "1433"
source_address_prefix = "10.0.1.0/24"
destination_address_prefix = "*"
resource_group_name   = azurerm_resource_group.rg.name
network_security_group_name = azurerm_network_security_group.nsg.name
}
// Associate NSG with subnet
resource "azurerm_subnet_network_security_group_association" "nsg_association" {
  subnet_id           = azurerm_subnet.subnet.id
  network_security_group_id = azurerm_network_security_group.nsg.id
}
// -----
// Public IPs for VMs
// -----
resource "azurerm_public_ip" "pip" {
  count                = var.vm_count
  name                = "pip-${count.index}"
  location            = azurerm_resource_group.rg.location
  resource_group_name = azurerm_resource_group.rg.name
  allocation_method   = "Static"
  sku                 = "Standard"
}
// -----
// Network Interfaces
// -----
resource "azurerm_network_interface" "nic" {
  count                = var.vm_count
  name                = "nic-${count.index}"
  location            = azurerm_resource_group.rg.location
  resource_group_name = azurerm_resource_group.rg.name

  ip_configuration {
    name                = "ipconfig-${count.index}"
    subnet_id          = azurerm_subnet.subnet.id
    private_ip_address_allocation = "Dynamic"
    public_ip_address_id = element(azurerm_public_ip.pip.*.id, count.index)
  }
}
// -----
// Virtual Machines (Windows Server 2019 Datacenter)
// -----
resource "azurerm_windows_virtual_machine" "vm" {

```

```

count          = var.vm_count
name           = "${var.vm_prefix}-${count.index}"
location       = azurerm_resource_group.rg.location
resource_group_name = azurerm_resource_group.rg.name
size          = count.index == 1 ? "Standard_B2s" : "Standard_B1s" # SQL = 2
CPU, others = 1 CPU
admin_username = var.admin_username
admin_password = var.admin_password

network_interface_ids = [
  element(azurerm_network_interface.nic.*.id, count.index)
]

os_disk {
  name          = "osdisk-${count.index}"
  caching       = "ReadWrite"
  storage_account_type = "Standard_LRS"
}

source_image_reference {
  publisher = "MicrosoftWindowsServer"
  offer     = "WindowsServer"
  sku       = "2019-Datacenter"
  version   = "latest"
}

tags = {
  Environment = "Production"
  Project     = "MTech-3VM"
  Role        = count.index == 0 ? "DomainController" : count.index == 1 ? "SQLServer" :
"PrintServer"
}
}

```

**** Variables.tf:**

```

// -----
// Variables Configuration
// -----
variable "resource_group_name" {
  description = "Name of the resource group"
  type        = string
}

```

```

    default      = "MTECH"
}
variable "location" {
    description = "Azure region for resources"
    type        = string
    default     = "Central India"
}
variable "vnet_name" {
    description = "Name of the virtual network"
    type        = string
    default     = "MTech_Network"
}
variable "subnet_name" {
    description = "Name of the subnet"
    type        = string
    default     = "subnet-project"
}
variable "vm_count" {
    description = "Number of virtual machines to create"
    type        = number
    default     = 3
}
variable "vm_size" {
    description = "Size of the virtual machines"
    type        = string
    default     = "Standard_B2s"
}
variable "admin_username" {
    description = "Administrator username for VMs"
    type        = string
    default     = "admin"
}
variable "admin_password" {
    description = "Administrator password for VMs"
    type        = string
    sensitive   = true
}
variable "vm_prefix" {
    description = "Prefix for VM names"
    type        = string
    default     = "mansivm"
}
variable "domain_name" {
    description = "Domain name for Active Directory"
    type        = string

```

```
    default      = "bits.com"
}

variable "domain_admin_username" {
  description = "Domain administrator username"
  type        = string
  default     = "DomainAdmin"
}

variable "domain_admin_password" {
  description = "Domain administrator password"
  type        = string
  sensitive   = true
}
```


Windows Server Update Services (WSUS):

To deploy software updates through Configuration Manager, we first need to install the WSUS (Windows Server Update Services) role. Later, WSUS will be connected with the Software Update Point (SUP) role in Configuration Manager.

WSUS works as a central tool that manages Microsoft updates for the organization. It downloads updates from Microsoft and helps distribute them to different computers across the network. This way, administrators can control which updates are installed and when, instead of every system downloading them individually.

After installing the WSUS role, we must configure where update files will be stored. This is done using the **post-install command**, which sets the content directory — the folder where WSUS will save downloaded updates.

Without this step, WSUS won't know where to store updates, and SCCM won't be able to deploy patches. It's like assigning a download folder before downloading anything — essential for proper operation.

```
PS C:\Users\Mansi> Install-WindowsFeature -Name UpdateServices, UpdateServices-Tools -IncludeManagementTools

Success Restart Needed Exit Code      Feature Result
-----
True      No          Success      {Windows Server Update Services, WSUS Serv...}
WARNING: Additional configuration may be required. Review the article Managing WSUS Using PowerShell at TechNet Library (http://go.microsoft.com/fwlink/?LinkId=235499) for more information on the recommended steps to perform WSUS installation using PowerShell.

PS C:\Users\Mansi> & 'C:\Program Files\Update Services\Tools\wsusutil.exe' postinstall SQL_INSTANCE_NAME="Mansim-1\SQLEXPRESS" CONTENT_I
Log file is located at C:\Users\Mansi\AppData\Local\Temp\2\WSUS_PostInstall_20251029T132135.log
Warning: The specified parameter SQL_INSTANCE_NAME has no effect because WSUS
is configured to use the Windows Internal Database.
Post install is starting
Post install has successfully completed
```

What is SCCM / MECM?

Microsoft Endpoint Configuration Manager (MECM) — previously known as SCCM or SMS — is a powerful tool from Microsoft used to manage large numbers of computers. It helps IT teams with:

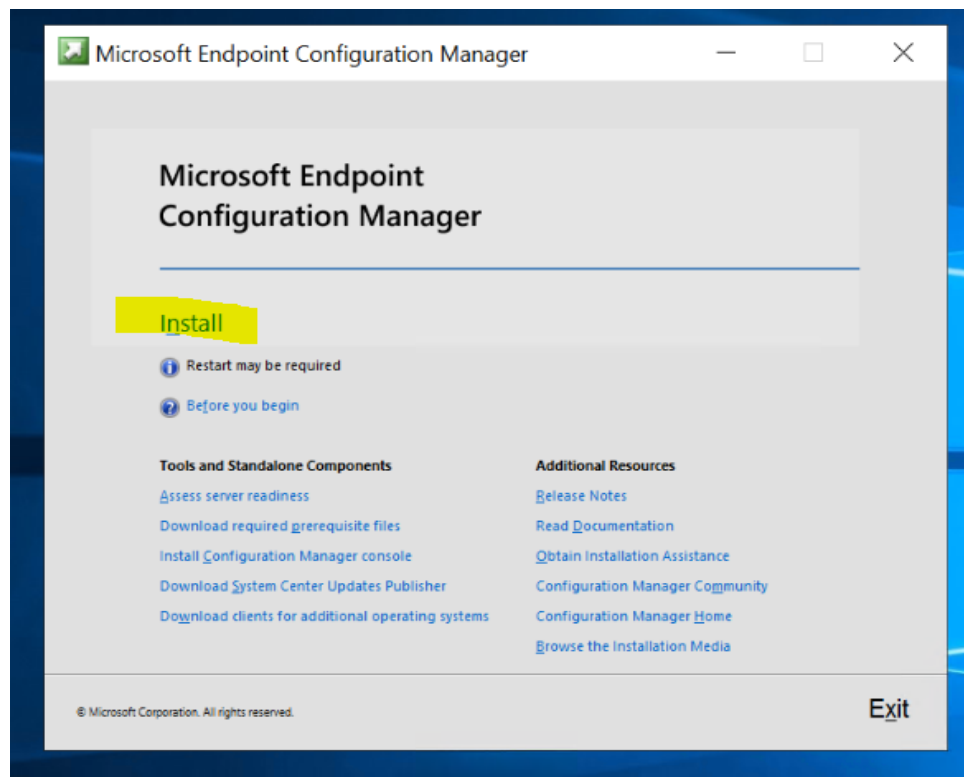
It helps organizations:

- Automatically deploy Windows updates and patches
- Install software remotely across hundreds or thousands of machines
- Roll out operating systems with zero-touch deployment
- Collect detailed hardware and software inventory
- Monitor compliance and system health

SCCM reduces manual effort, improves security, and ensures consistency across all managed devices. It's widely used in enterprises to maintain control over their IT infrastructure.

SCCM Installation Made Easy

The below screenshots will be help for easy installation of SCCM in the server. Anyone can install by following the screenshots.



Microsoft Endpoint Configuration Manager Setup Wizard

Before You Begin

This wizard walks you through the steps necessary to install or upgrade Configuration Manager. Setup also provides you with options to recover a site, perform site maintenance, and uninstall the site.

Before you run this wizard:

1. Verify that you have a supported Microsoft SQL Server installation available for Configuration Manager.
2. Identify the FQDN of the computer that is running Microsoft SQL Server.
3. Confirm that the computers for Configuration Manager site systems meet the minimum system requirements.
4. Read the [release notes online](#) for important information and for links to supported versions and system requirements.

WARNING: This program is protected by copyright law and international treaties.

Unauthorized reproduction or distribution of this program, or any portion of it, may result in severe civil and criminal penalties, and will be prosecuted to the maximum extent possible under law.

< Previous **Next >** Cancel

Microsoft Endpoint Configuration Manager Setup Wizard

Getting Started

Available Setup Options

Setup has not detected an existing installation of site server, site system, or Configuration Manager console on this computer.

☒ **Install a Configuration Manager primary site**

- ☐ Use typical installation options for a stand-alone primary site
 - Install a Configuration Manager primary site
 - Use default installation path
 - Configure local SQL Server with default settings
 - Enable a local management point for Configuration Manager
 - Enable a local distribution point for Configuration Manager
- ☐ Install a Configuration Manager central administration site
- ☐ Upgrade this Configuration Manager site
- ☐ Recover a site
- ☐ Perform site maintenance or reset this site
- ☐ Uninstall this Configuration Manager site

< Previous **Next >** Cancel

Microsoft Endpoint Configuration Manager Setup Wizard

Product Key

☒ Install the evaluation edition of this product
When you install the Current Branch evaluation edition of this product, it is fully functional for 180 days.

☐ Install the licensed edition of this product

I acknowledge that I currently have an active Software Assurance license agreement with Microsoft. I understand that this version of Configuration Manager will have regular updates that can include new feature offerings.

Software Assurance expiration date. This date must be after October 1st, 2016:

Select a date:

[Learn more](#)

< Previous Next > Cancel

Microsoft Configuration Manager Setup Wizard

Prerequisite Downloads

Setup requires prerequisite files. Setup can automatically download the files to a location that you specify, or you can use files that have been downloaded previously.

☒ Download required files

Example: \\ServerName\ShareName or C:\Downloads

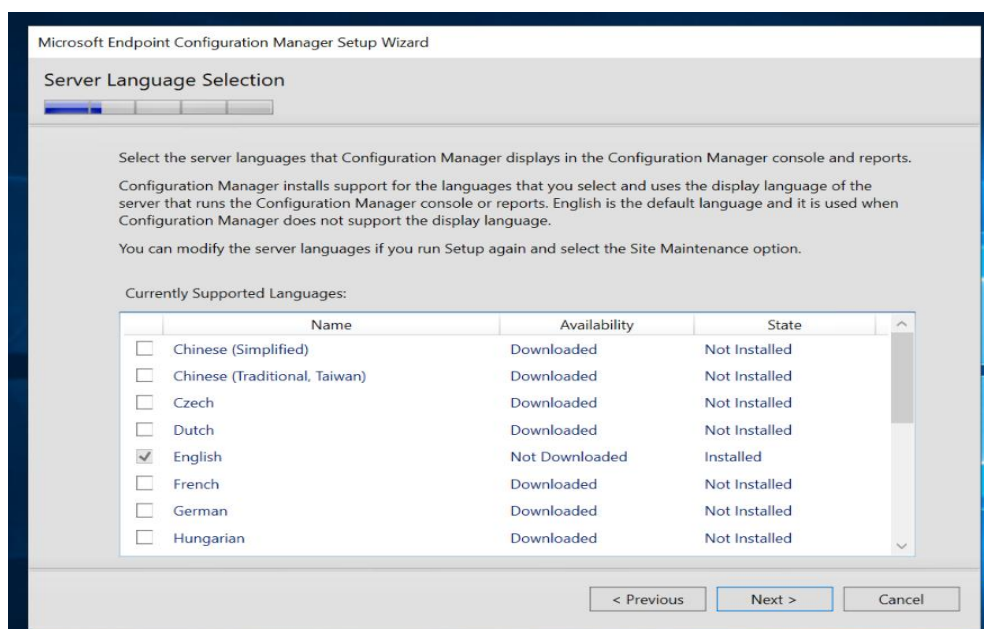
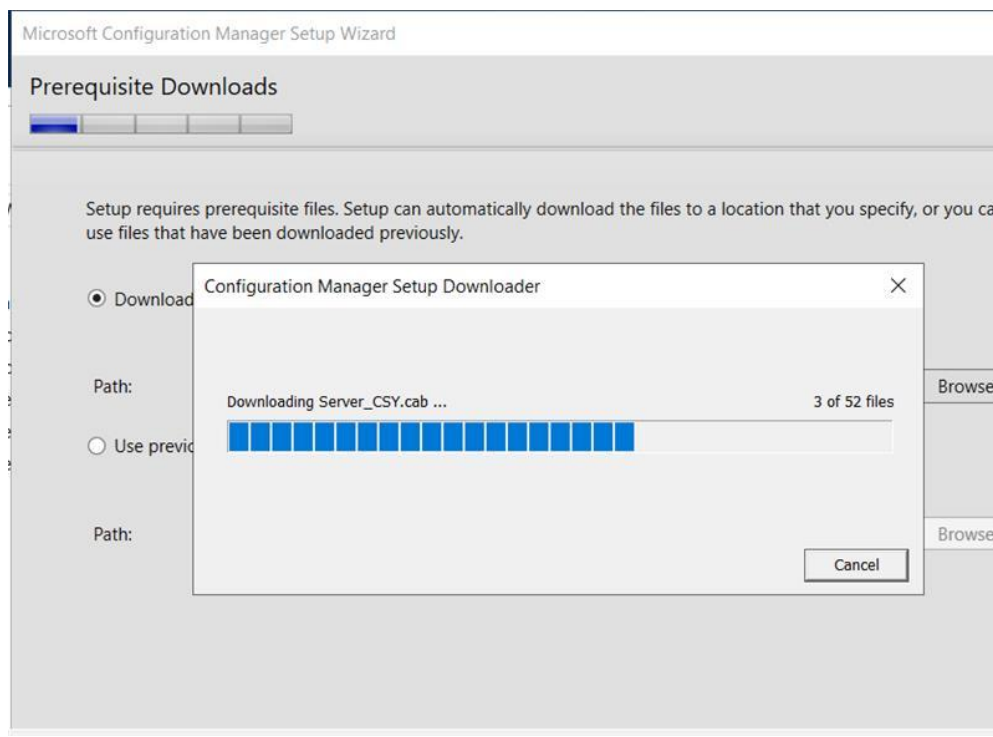
Path:

☐ Use previously downloaded files

Example: \\ServerName\ShareName or C:\Downloads

Path:

< Previous Next > Cancel



Microsoft Configuration Manager Setup Wizard

Site and Installation Settings

Specify a site code that uniquely identifies this Configuration Manager site in your hierarchy.

Site code:

Specify a site name that helps to identify the site. Example: Contoso Headquarters Site

Site name:

Note: The site code must be unique in the Configuration Manager hierarchy and cannot be changed after you install the site.

Installation folder:

Specify whether to install the Configuration Manager console to manage the Configuration Manager site from this computer. You can remotely manage the site when you do not install the Configuration Manager console.

☒ Install the Configuration Manager console

< Previous Next > Cancel

Microsoft Endpoint Configuration Manager Setup Wizard

Primary Site Installation

Specify whether to join the primary site to an existing Configuration Manager hierarchy or install the primary site as a stand-alone site.

☐ Join the primary site to an existing hierarchy

Central administration site server (FQDN): Example: server1.contoso.com

☒ Install the primary site as a stand-alone site

< Previous Next > Cancel

Database Information

Configuration Manager primary sites require a Microsoft SQL Server database to store site settings and data.

Specify the site database server details. The instance name that you use for the site database must be configured with a static TCP port. Dynamic ports are not supported.

SQL Server name (FQDN): Example: Server1.contoso.com

mansivm-1.bits.com

Instance name (leave blank for default): Example: MyInstance

Database name: Example: CM_XYZ

CM_BIT

Specify the TCP port number for SQL Server Service Broker. Configuration Manager uses Service Broker to replicate data between parent and child site database servers in the hierarchy. This port is different from the port used by the SQL Server service, which is automatically detected by Configuration Manager.

Service Broker Port:

4022

< Previous

Next >

Cancel

Microsoft Endpoint Configuration Manager Setup Wizard

Database Information

Specify the locations for the SQL Server data file and transaction log file.

Path to the SQL Server data file

C:\Program Files\Microsoft SQL Server\MSSQL13.MSSQLSERVER\MSSQL\DATA

Path to the SQL Server log file

C:\Program Files\Microsoft SQL Server\MSSQL13.MSSQLSERVER\MSSQL\DATA

< Previous Next > Cancel

Microsoft Configuration Manager Setup Wizard

SMS Provider Settings

SMS Providers are used by the Configuration Manager console to communicate with the site database.

Specify the server where the SMS Provider will be installed.

SMS Provider (FQDN): Example: server1.contoso.com

mansivm-1.bits.com

Note: The SMS Provider cannot be installed on a server that is configured for SQL Server clustering.

< Previous Next > Cancel

Microsoft Endpoint Configuration Manager Setup Wizard

Client Computer Communication Settings

Configuration Manager site system roles can accept HTTP or HTTPS communication from clients. Specify whether to require all site system roles to accept only HTTPS communication or allow the communication method to be configured on each site system role.

☒ All site system roles accept only HTTPS communication from clients

☐ Configure the communication method on each site system role

☐ Clients will use HTTPS when they have a valid PKI certificate and HTTPS-enabled site roles are available

Note: HTTPS communication requires client computers to have a valid PKI certificate for client authentication.

< Previous Next > Cancel

Microsoft Endpoint Configuration Manager Setup Wizard

Client Computer Communication Settings

Configuration Manager site system roles can accept HTTP or HTTPS communication from clients. Specify whether to require all site system roles to accept only HTTPS communication or allow the communication method to be configured on each site system role.

☒ All site system roles accept only HTTPS communication from clients

☐ Configure the communication method on each site system role

☐ Clients will use HTTPS when they have a valid PKI certificate and HTTPS-enabled site roles are available

Note: HTTPS communication requires client computers to have a valid PKI certificate for client authentication.

< Previous Next > Cancel

Microsoft Configuration Manager Setup Wizard

Site System Roles

Specify whether to have Setup install a management point or distribution point.

A management point provides clients with policy and content location information. It also receives configuration data from clients.

☒ Install a management point.

FQDN: Client connection:

A distribution point contains source files for clients to download and lets you control content distribution by using bandwidth, throttling, and scheduling controls.

☒ Install a distribution point.

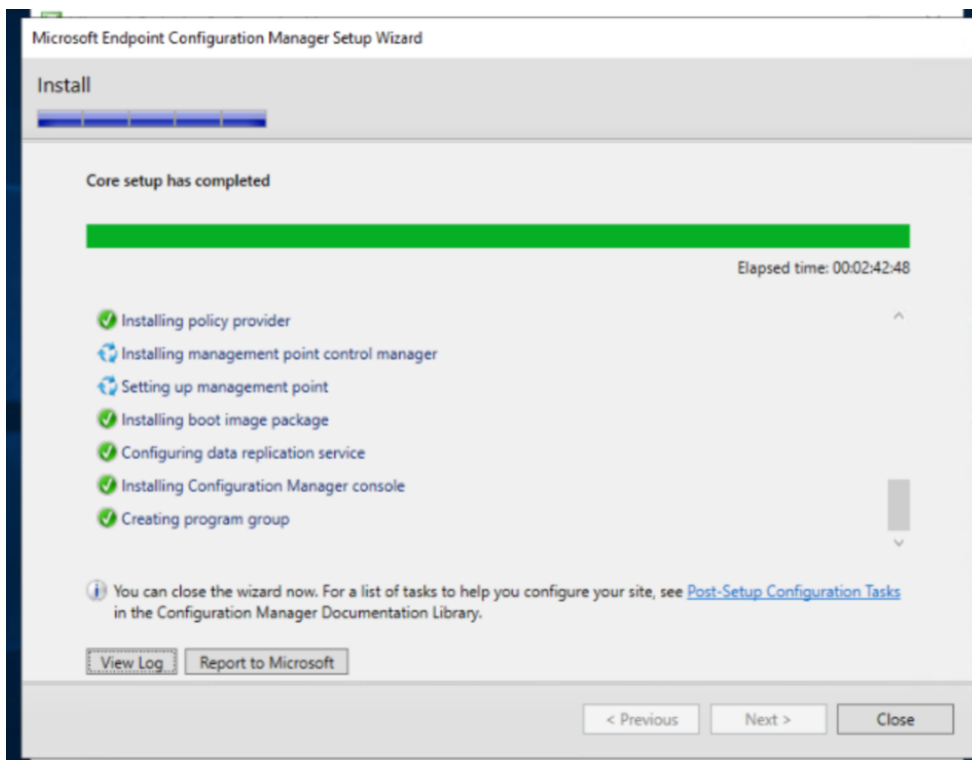
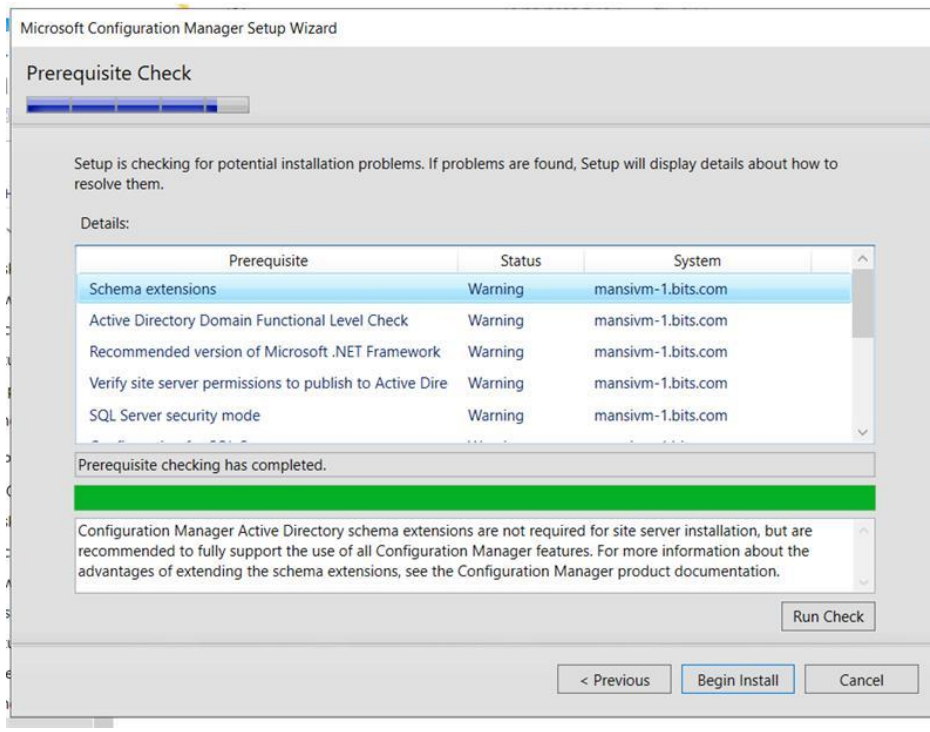
FQDN: Client connection:

The site server's computer account is used to install the selected site system roles. Ensure that this account is a member of the local administrators group for the specified servers.

You can install additional site system roles from the Configuration Manager console after Setup finishes.

Site system roles configured to use HTTPS must have a valid PKI server certificate.

< Previous Next > Cancel



• SCCM Configuration After Installation

After successfully installing SCCM, I proceeded to configure the environment by integrating it with Active Directory. This allowed SCCM to discover and manage users, computers, and groups within the domain.

◆ Primary Site Setup

I configured the SCCM primary site using the following details:

- **Site Code: BIT**
- **Site Name: Bits**

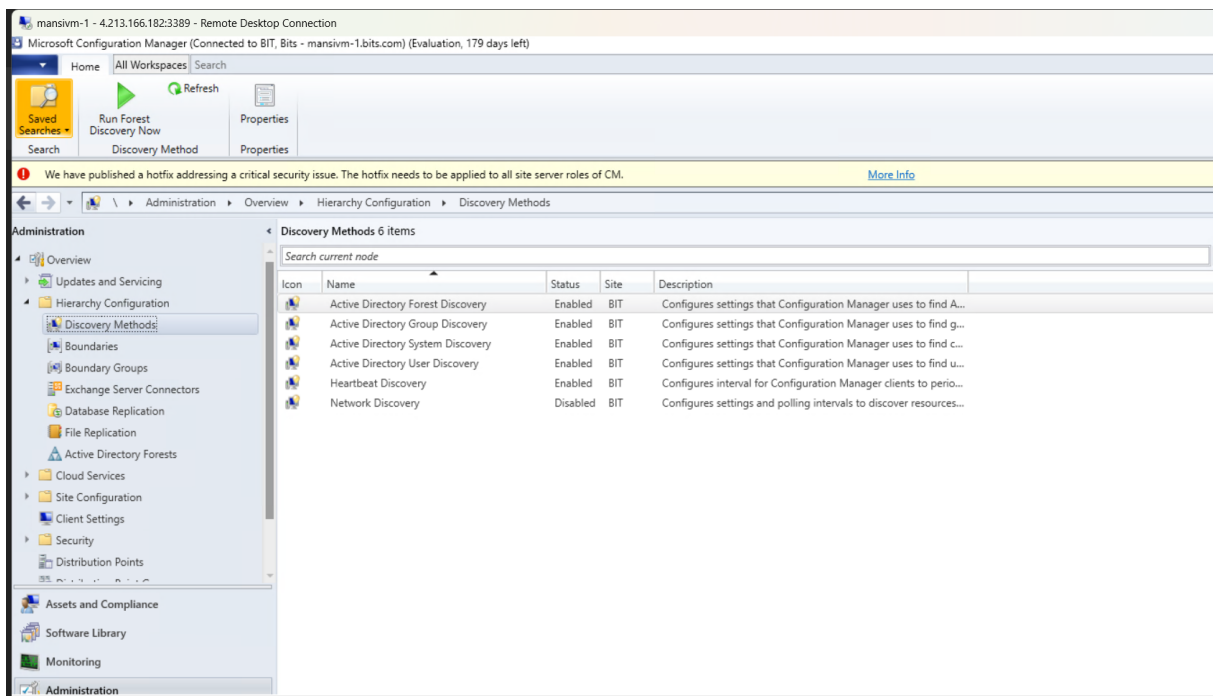
This site now serves as the central management point for all SCCM operations including deployments, patching, and inventory.

Active Directory Integration:

To enable SCCM to fetch data from AD, I activated the following discovery methods:

- **Active Directory System Discovery** – to detect computers
- **Active Directory User Discovery** – to detect user accounts
- **Active Directory Group Discovery** – optional, for AD groups

Each method was configured with the appropriate LDAP path, and a full discovery was initiated to ensure complete synchronization.

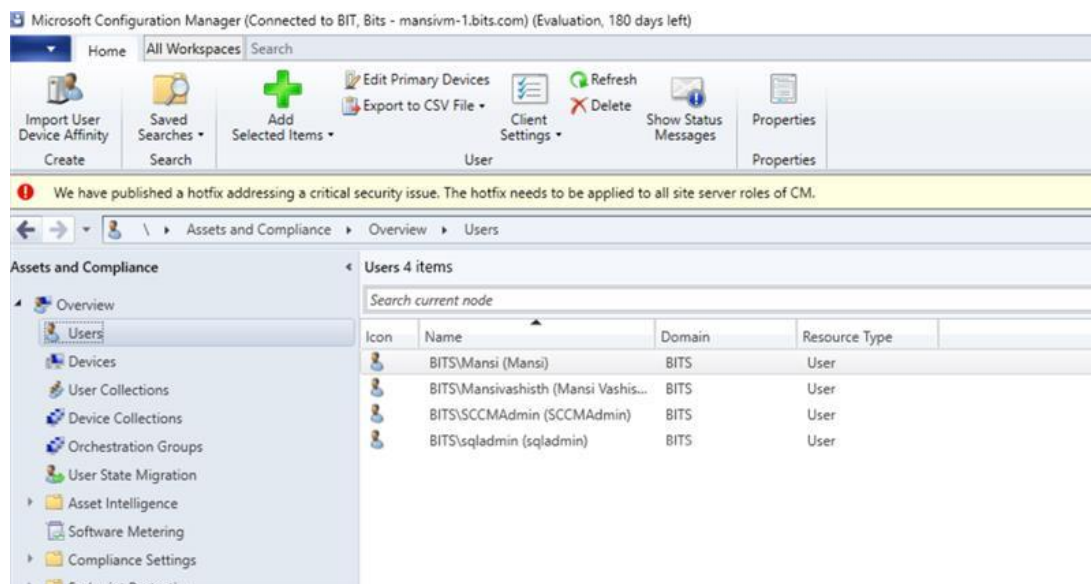
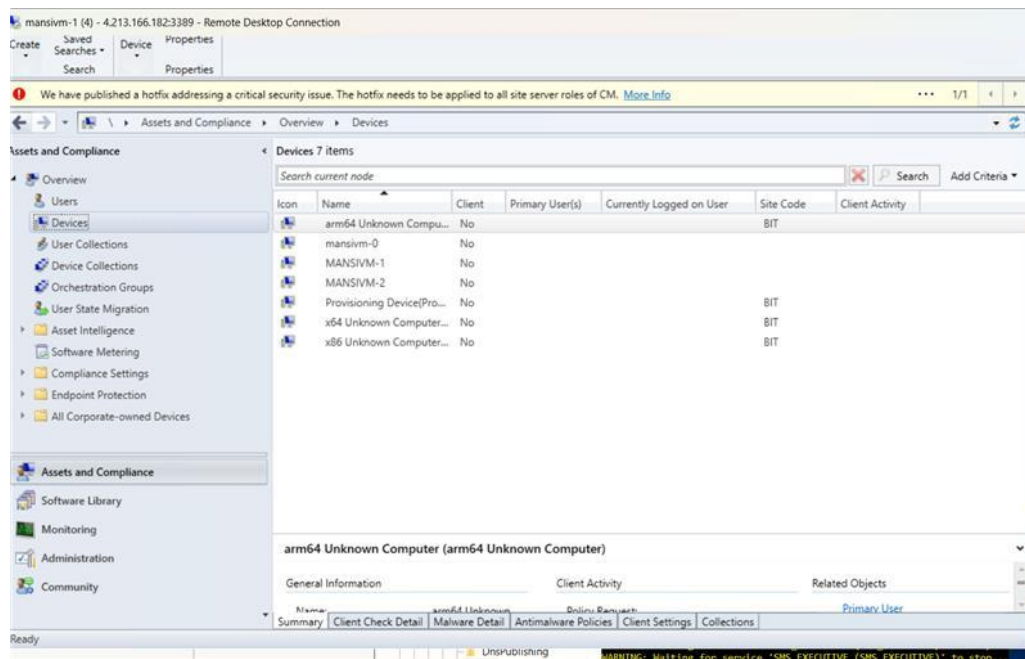


Verification:

Post-discovery, I verified that:

- Devices and users appeared under Assets and Compliance
- SCCM was successfully communicating with AD

This confirmed that the SCCM environment was fully integrated and ready for client management and software deployment.

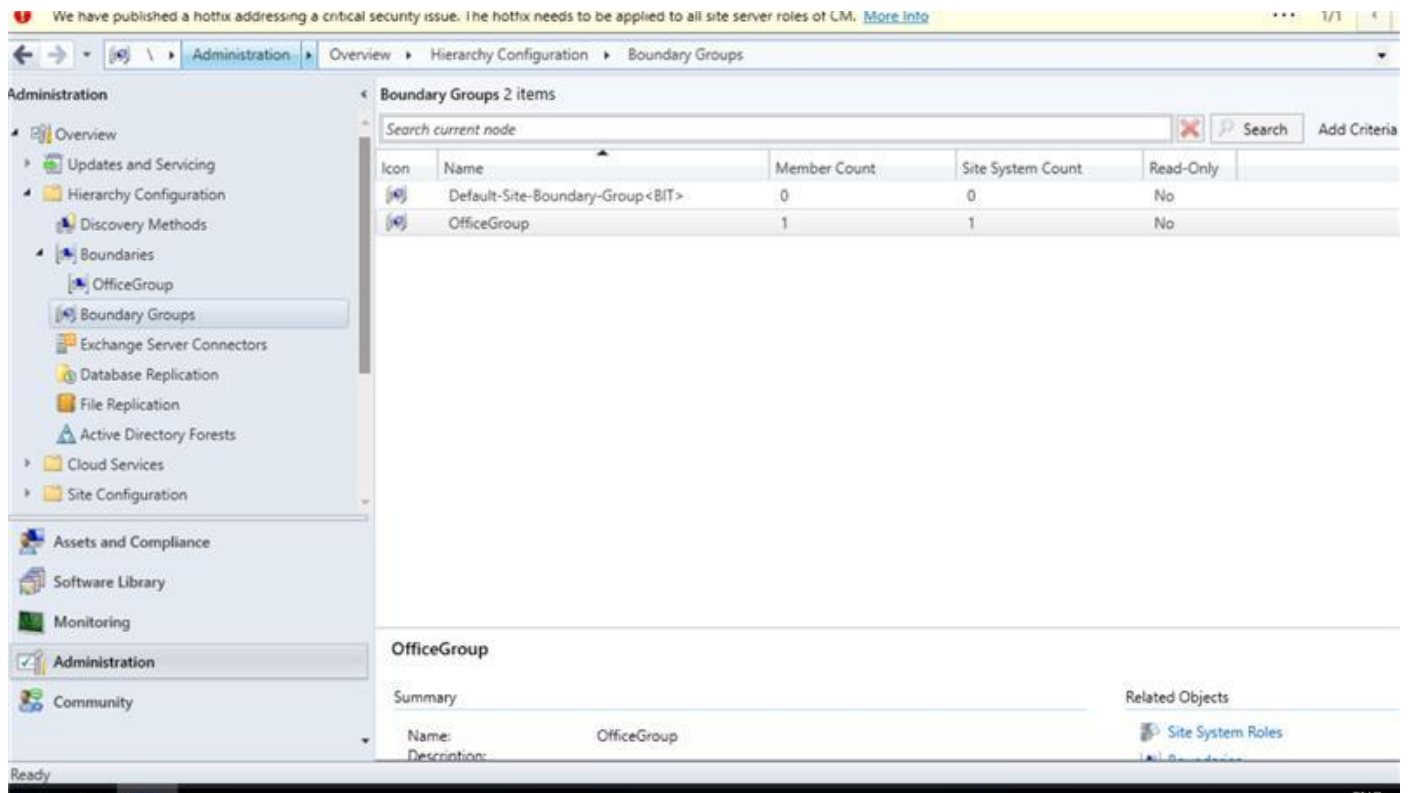


Boundary Group Configuration and Client Installation

After configuring the SCCM site and integrating Active Directory, I proceeded to set up boundary groups to ensure proper site assignment and content distribution.

◆ Boundary Group Setup

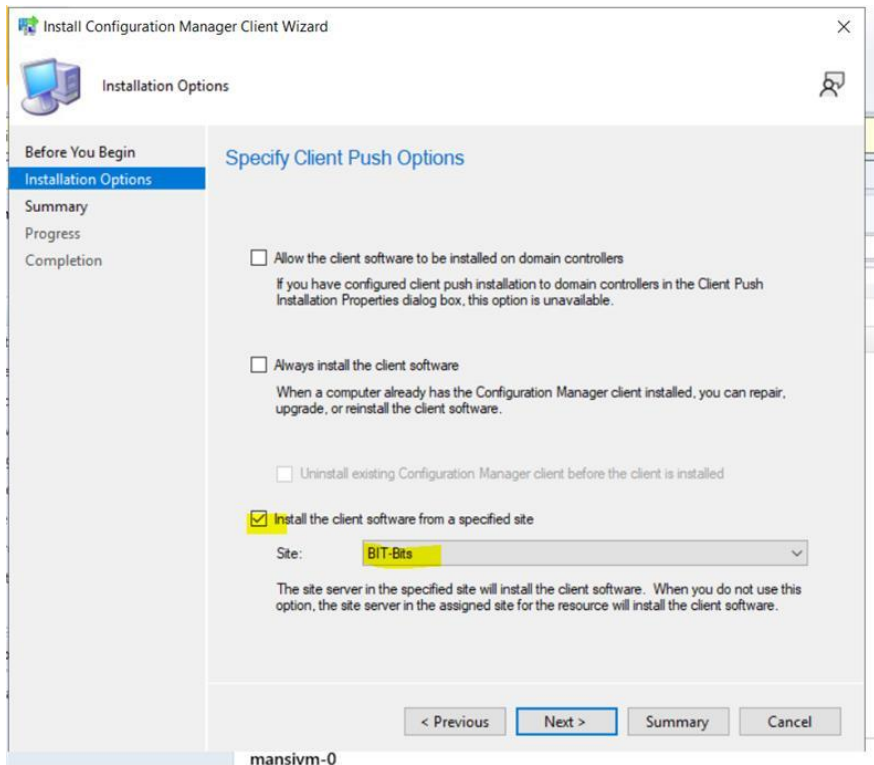
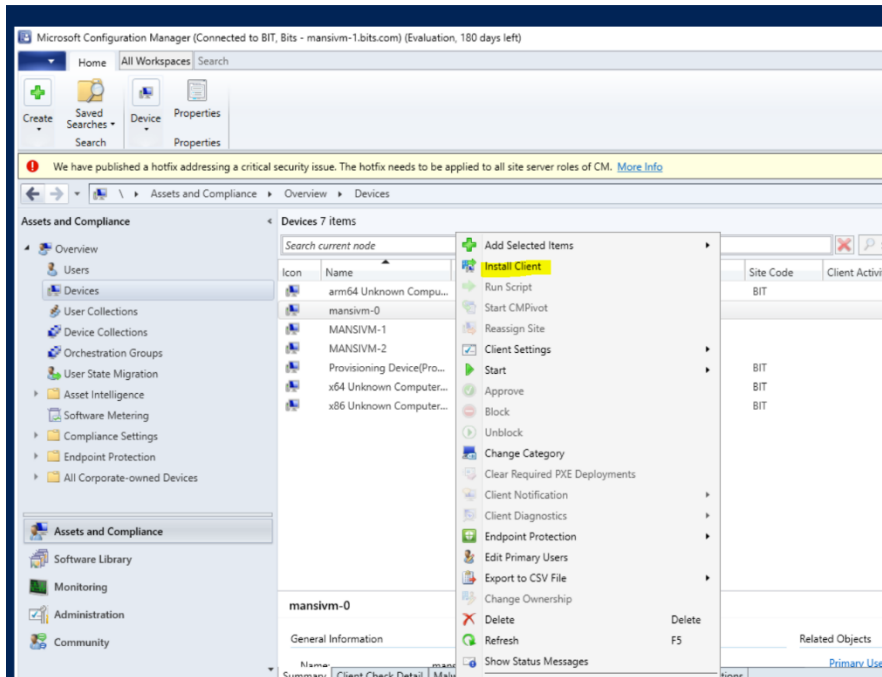
- Created a boundary group and added the relevant subnet to define the network location.
- Linked the boundary group to the primary site (BIT) under the **References** tab.
- Verified that the site system hosting the **Management Point** and **Distribution Point** was added.
- Enabled options for site assignment and content location to ensure clients could receive updates and applications.

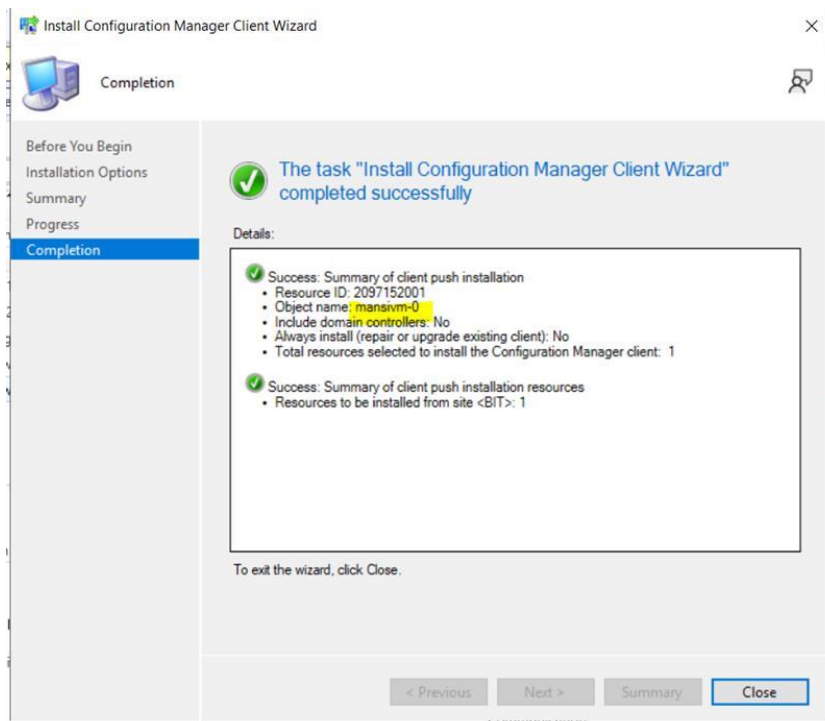


Client Push Installation

Once discovery was complete and boundary groups were properly configured:

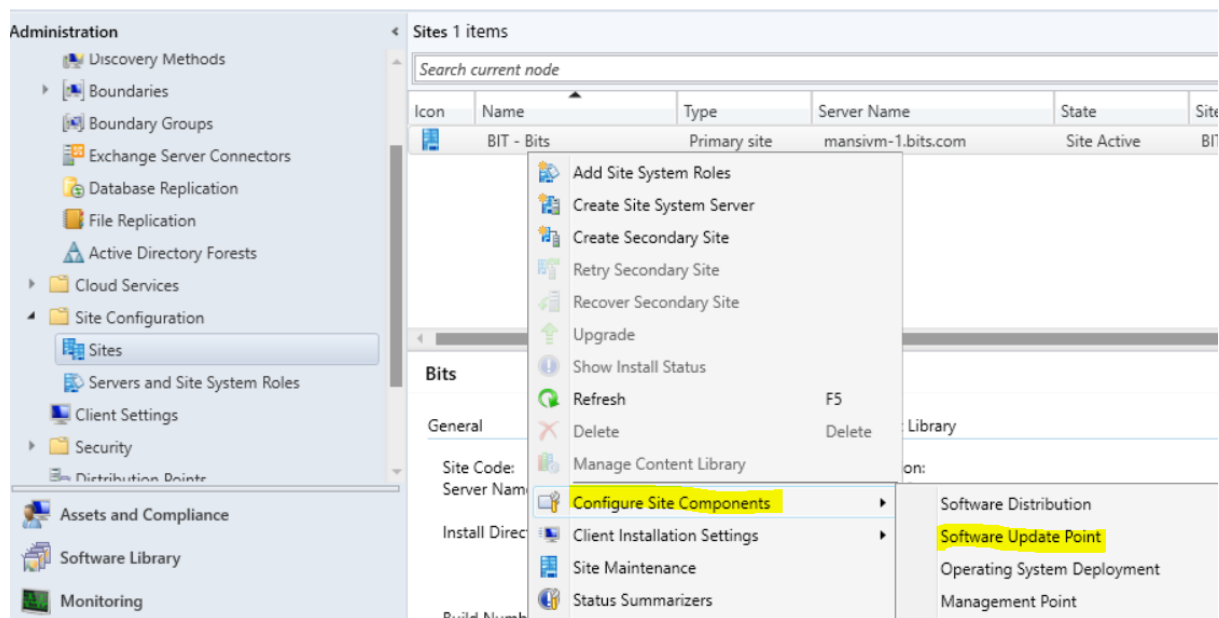
- Initiated client push installation for three servers listed under **Assets and Compliance** → **Devices**.

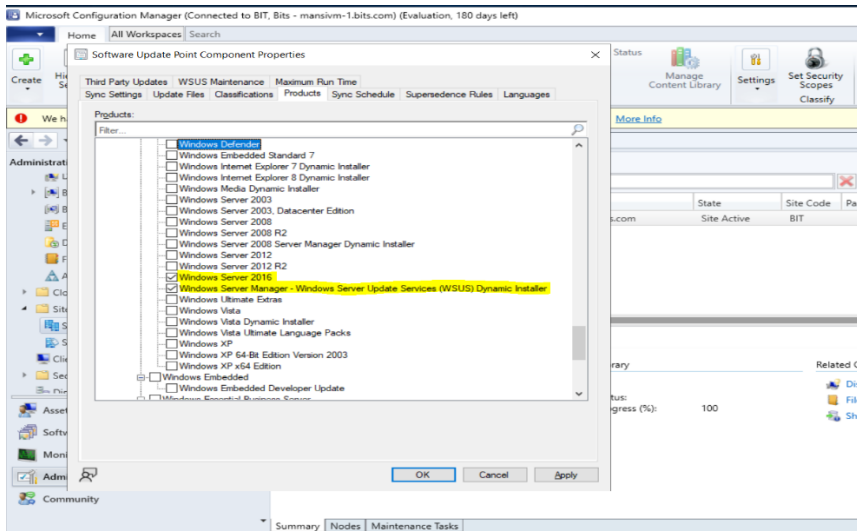




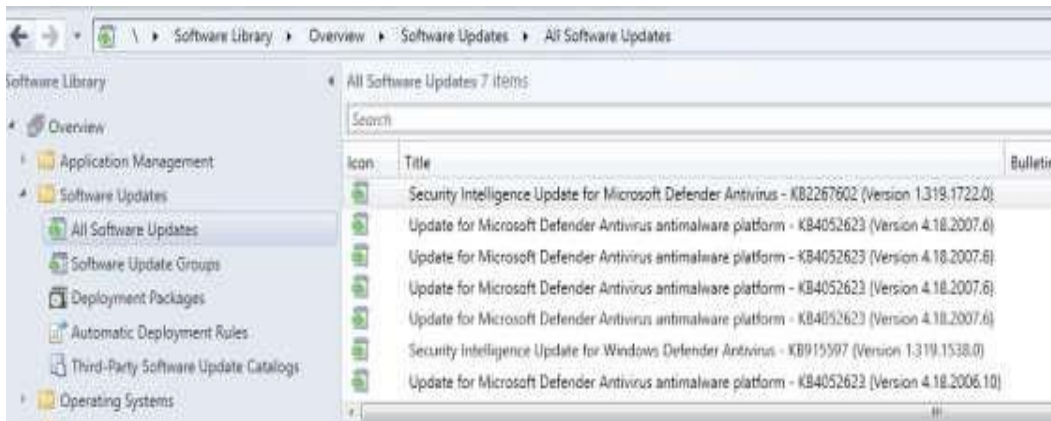
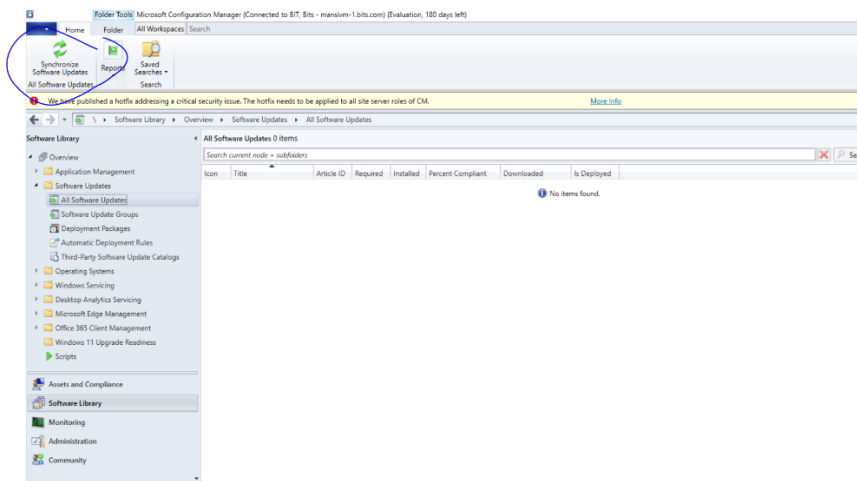
Synchronizing and importing updates from Microsoft site:

- Added the Software Update Point role to the SCCM site server





• Synchronization and Verification:



RDS FARM

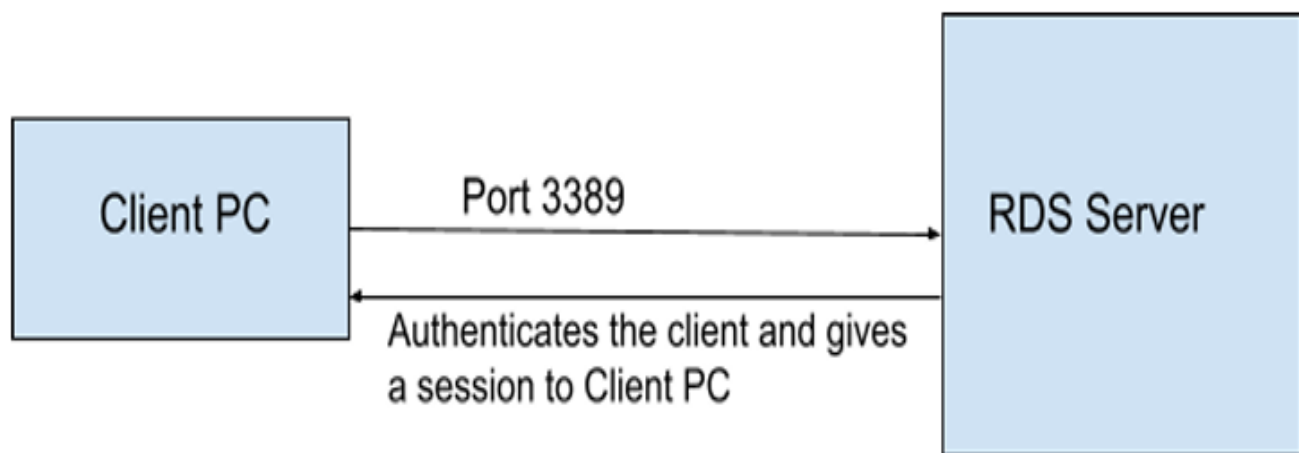
An RDS farm is a scalable setup of Remote Desktop Services where multiple servers work together to provide secure, load-balanced remote access to users. It includes roles like RD Gateway, Connection Broker, Session Hosts, and Web Access — each with a specific job.

RDS is a role, which is found in windows server applications which gives RD sessions, apps, VMs and is managed by the central server. RDS Farm is a group of session host servers and a connection broker which helps users to connect to the session on the servers. In an azure environment we have more advantage of cost effectiveness, hardware independence, reliability, scalability and secure remote access.

It uses RDP(Remote desktop Protocol) which uses port 3389 for connection to session host servers. A client can connect to the machine from their PC using RDP and can do their daily tasks.

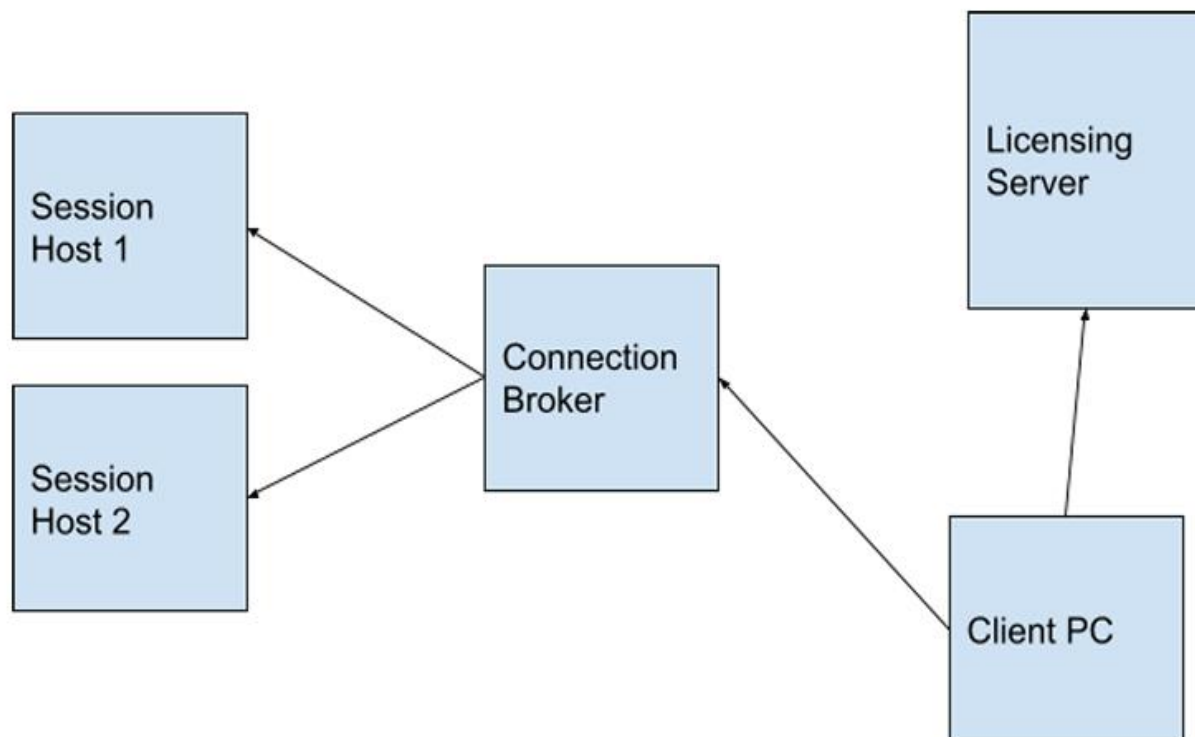
Reasons for RDP

- It's independent of Hardware's.
- No issues of incompatibility of applications.
- Reduces cost of Hardware as it does not require separate hardware setup for each user. Users can connect from their devices.
- Incompatible OS architecture



Components of an RDS Farm

Component	What It Does
RD Session Host	Hosts the actual desktop sessions or Remote Apps. Users connect here.
RD Connection Broker	Manages user sessions and redirects users to available Session Hosts.
RD Gateway	Allows secure access from outside the network using HTTPS (port 443).
RD Web Access	Provides a web portal for users to launch RemoteApps or desktops.
RD Licensing	Manages RDS licenses (optional if using Per User mode for small setups).



- **RD Session Host :**

This is the server where users actually work. It runs the remote desktop or RemoteApp session. You can have multiple Session Hosts in a farm to balance the load and avoid performance issues. In my case, mansivm-0 and mansivm-1 are session host.

- **RD Connection Broker:**

This is the brain of the RDS farm. It tracks all active sessions and decides which Session Host a user should connect to. If a user disconnects and reconnects, the broker ensures they return to the same session. It also handles load balancing and redirection.

- **RD Gateway:**

This role allows users to connect securely from outside the network using HTTPS (port 443). It wraps the RDP traffic inside SSL, so users don't need a VPN. It's perfect for remote access from home or client locations. In my case, Mansivm-2 is RD Gateway.

- **RD Web Access**

This is the web-based portal where users can log in and launch RemoteApps or full desktops. It's useful for non-technical users who prefer clicking icons instead of using mstsc. In Short, pprovides a browser-based portal where users can launch RemoteApps or full desktops. It's the user-friendly front end.

- **RD Licensing**

This role manages RDS license usage. In small setups, you can use **Per User mode** without buying CALs. For larger environments, you need to install and activate the RD Licensing Server.

- **Session Collection**

A Session Collection is a group of Session Hosts that serve a specific set of users or applications. You can create multiple collections for different departments or use cases. It helps organize and isolate workloads. In my case I have named my collection -

RDSCollection

Step by step commands performed to setup RDS Farm:

```
#Server Roles

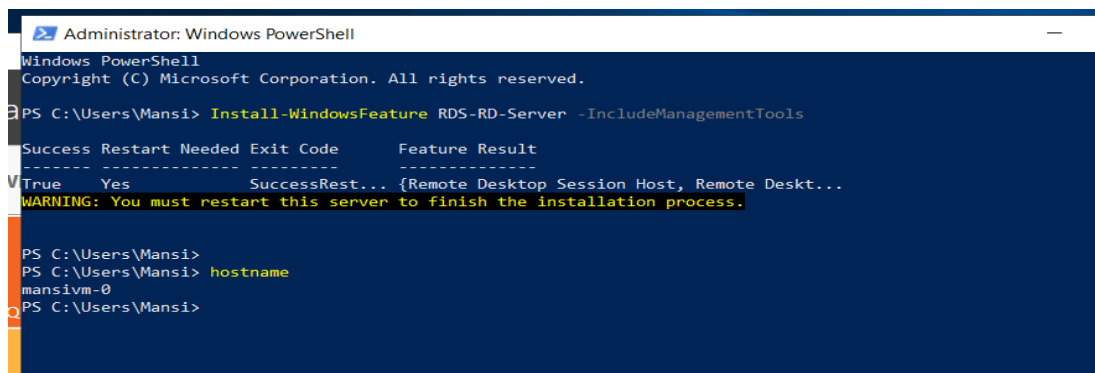
$ConnectionBroker = "mansivm-2.bits.com"
$WebAccessServer = "mansivm-2.bits.com"
$RDGateway = "mansivm-2.bits.com"
$SessionHosts = @("mansivm-0.bits.com", "mansivm-1.bits.com")
$CollectionName = "RDS_Collection"
```

- Step 1: Login to all three VM and run below commands to enable PowerShell remoting and enable firewall rules"

```
Enable-PSRemoting -Force
Start-Service WinRM
Set-Item WSMan:\localhost\Client\TrustedHosts -Value "*" -Force
winrm quickconfig
Enable-NetFirewallRule -Name "WINRM-HTTP-In-TCP"
Enable-NetFirewallRule -DisplayGroup "Remote Desktop"
```

- Step 2: Login to Mansivm-0 and Mansivm-1 to install session host:

Install-WindowsFeature RDS-RD-Server -IncludeManagementTools



```
Administrator: Windows PowerShell
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\Mansi> Install-WindowsFeature RDS-RD-Server -IncludeManagementTools

Success Restart Needed Exit Code      Feature Result
-----
True    Yes          SuccessRest... {Remote Desktop Session Host, Remote Deskt...
WARNING: You must restart this server to finish the installation process.

PS C:\Users\Mansi>
PS C:\Users\Mansi> hostname
mansivm-0
PS C:\Users\Mansi>
```

- Step 3: Login to Mansivm-2 to proceed futher and setup farm:

```
# --- Define server roles ---
$ConnectionBroker = "mansivm-2.bits.com"
$WebAccessServer = "mansivm-2.bits.com"
$RDGateway = "mansivm-2.bits.com"
$SessionHosts = @("mansivm-0.bits.com", "mansivm-1.bits.com")
$CollectionName = "RDS_Collection"

# --- Deploy RDS farm with session hosts ---
New-RDSessionDeployment -ConnectionBroker $ConnectionBroker -WebAccessServer
$WebAccessServer -SessionHost $SessionHosts

# --- Create and bind self-signed cert for Gateway ---
$cert = New-SelfSignedCertificate -DnsName $RDGateway -CertStoreLocation
"Cert:\LocalMachine\My"
Set-RDCertificate -Role RDGateway -Thumbprint $cert.Thumbprint -ConnectionBroker
$ConnectionBroker -Force
Set-RDCertificate -Role RDWebAccess -Thumbprint $cert.Thumbprint -ConnectionBroker
$ConnectionBroker -Force

# --- Set licensing mode to PerUser (no CALs needed) ---
Set-RDLicenseConfiguration -LicenseServer $ConnectionBroker -Mode PerUser

# --- Create session collection ---
New-RDSessionCollection -CollectionName $CollectionName -SessionHost $SessionHosts -
ConnectionBroker $ConnectionBroker

# --- Configure load balancing to fix farm redirection ---
Set-RDSessionCollectionConfiguration -CollectionName $CollectionName `
  -CustomRdpProperty "loadbalanceinfo:s:tsv://MS Terminal Services
Plugin.1 $CollectionName"

# --- Configure RD Gateway ---
Set-RDDeploymentGatewayConfiguration `
  -GatewayMode Custom `
  -GatewayExternalFqdn $RDGateway `
  -LogonMethod AllowUserToSelectDuringConnection `
  -UseCachedCredentials $true

# --- Final message ---
Write-Host " RDS setup complete."
```

****Validate RDS Collection group status and members:**

```
PS C:\Windows\system32> Get-RDSessionCollection -ConnectionBroker "mansivm-2.bits.com"
```

CollectionName	Size	ResourceType	CollectionType	CollectionDescription
RDSCollection	2	Remote Desktop	PooledUnmanaged	Main RDS Session Collection

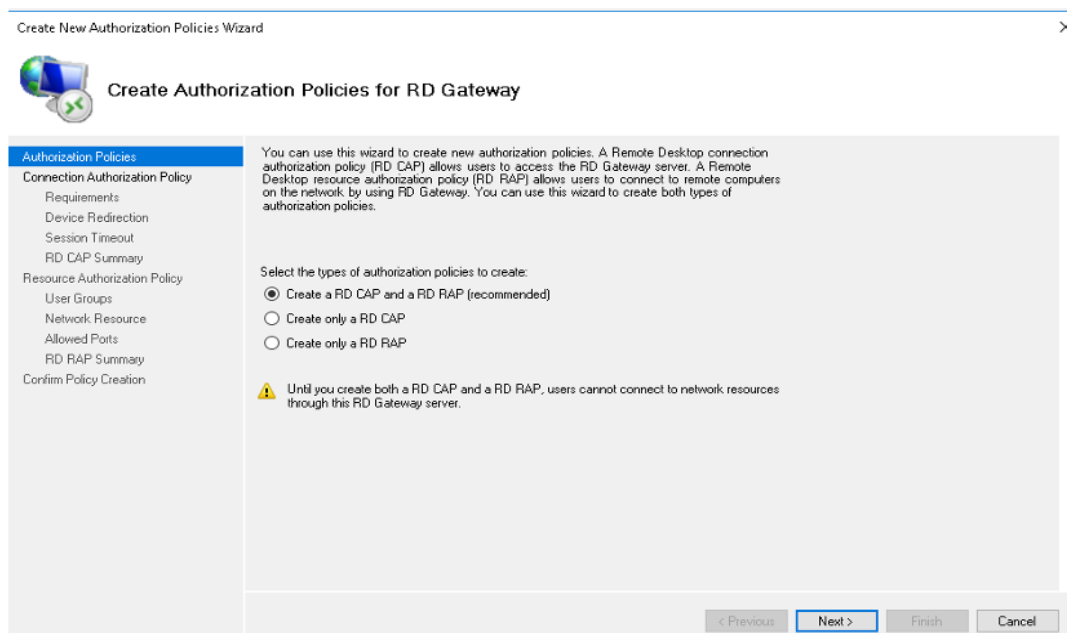
```
PS C:\Windows\system32> Get-RDSessionHost -CollectionName "RDSCollection" -ConnectionBroker "mansivm-2.bits.com"
```

CollectionName	SessionHost	NewConnectionAllowed
RDSCollection	mansivm-1.bits.com	Yes
RDSCollection	MANSIVM-0.BITS.COM	Yes

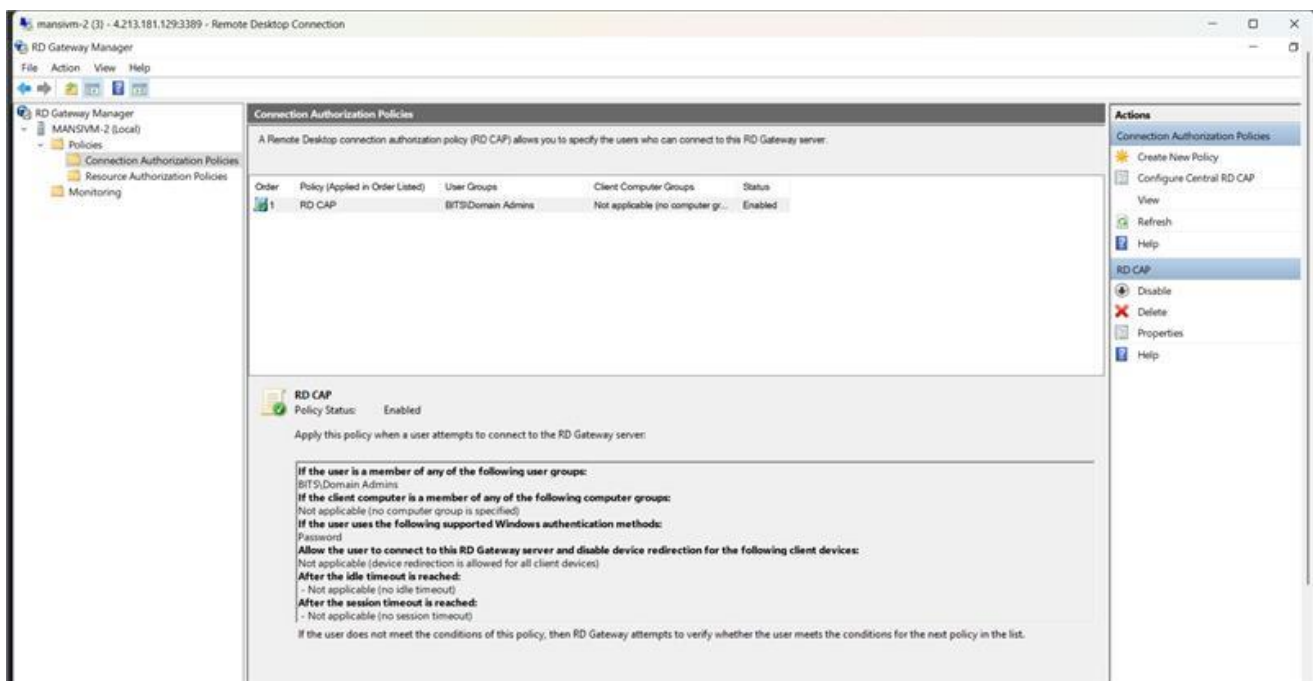
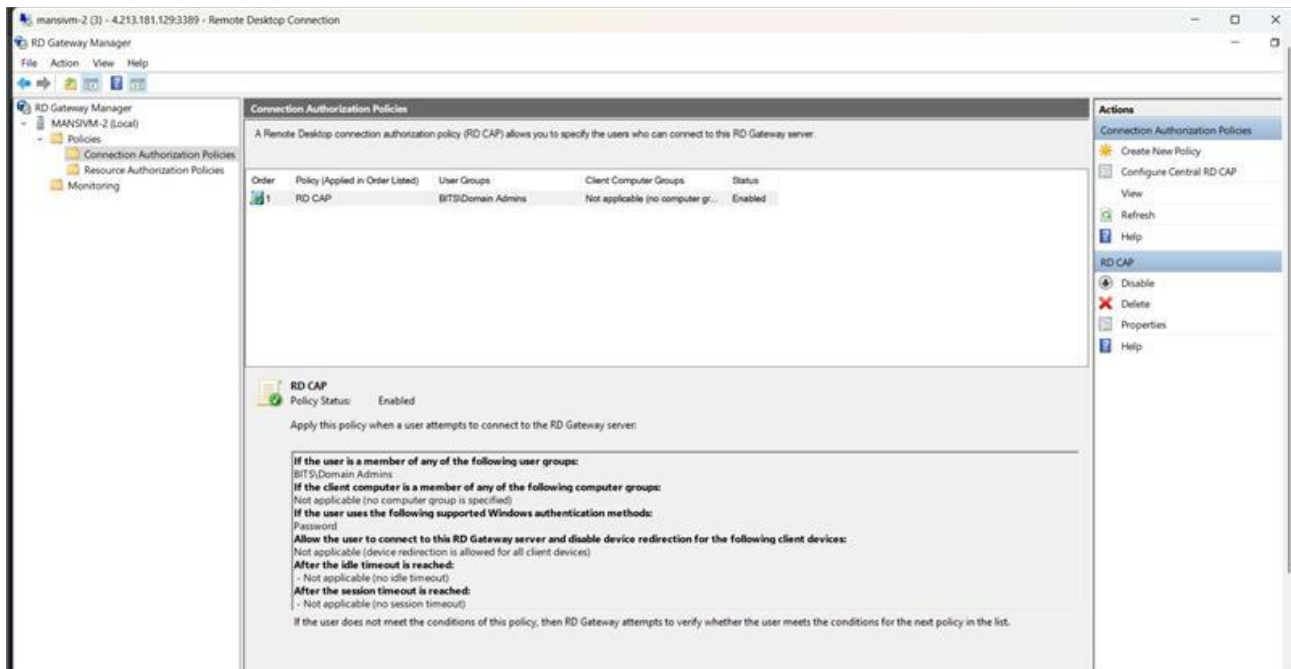
```
PS C:\Windows\system32>
```

**** Creating Authorization policy for RD Gateway****

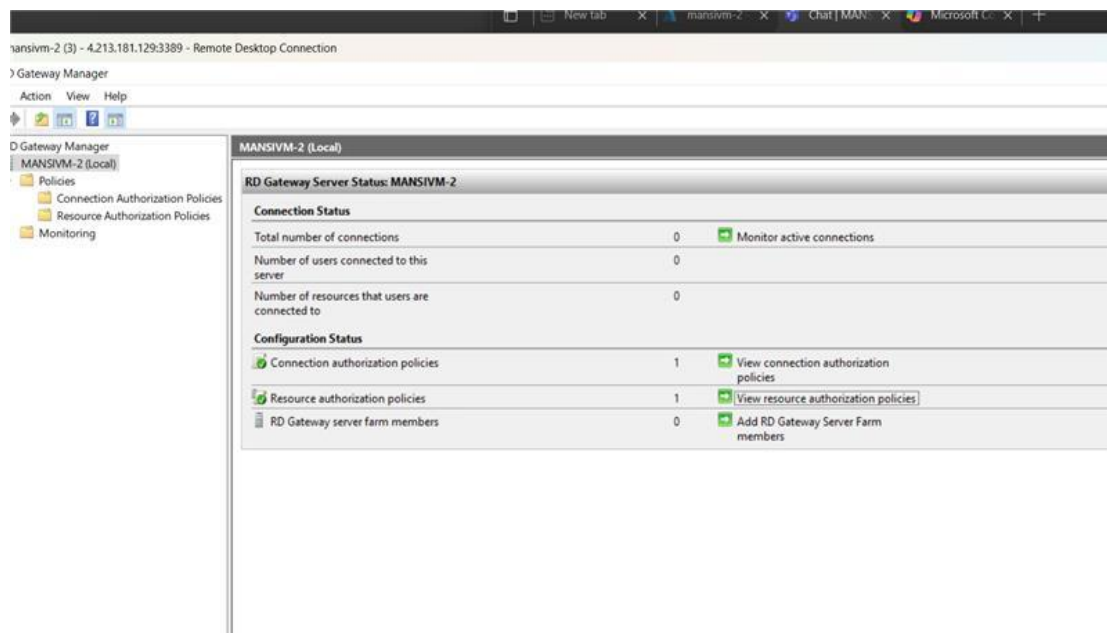
a. We will open Remote Desktop Gateway and the click on creating Authorization policy.



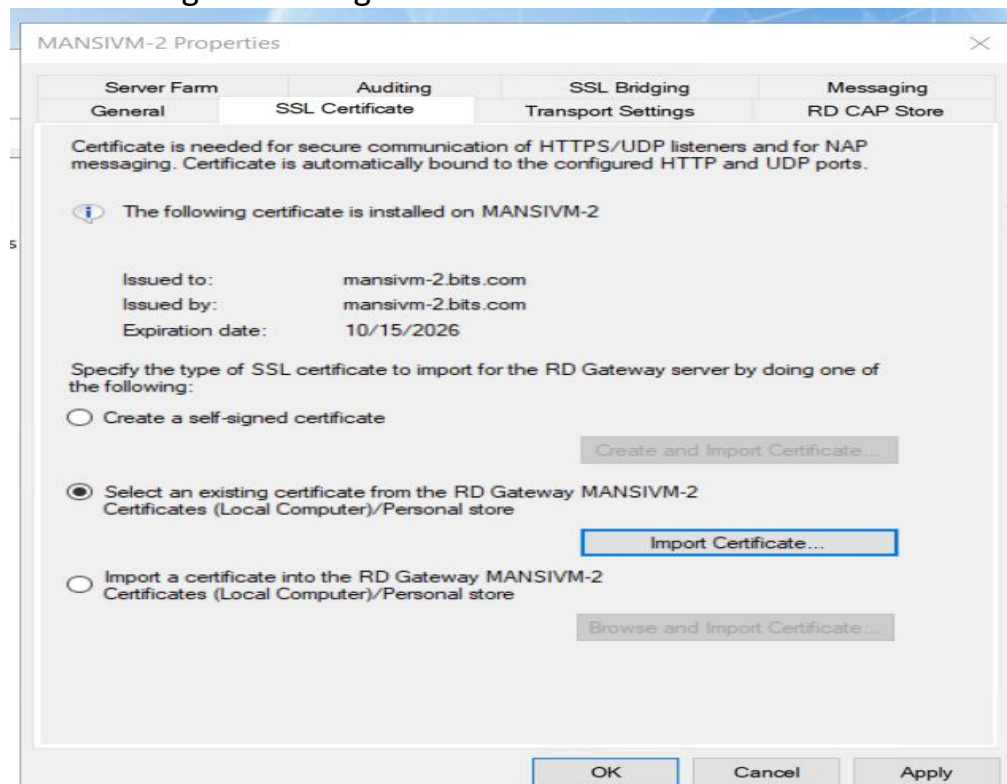
b. We will create both Connection Authorization Policy(CAP) and Resource Authorization Policy(RAP). First RD CAP will be created.



C. Once both policy will be created our RDGateway setup will be completed.

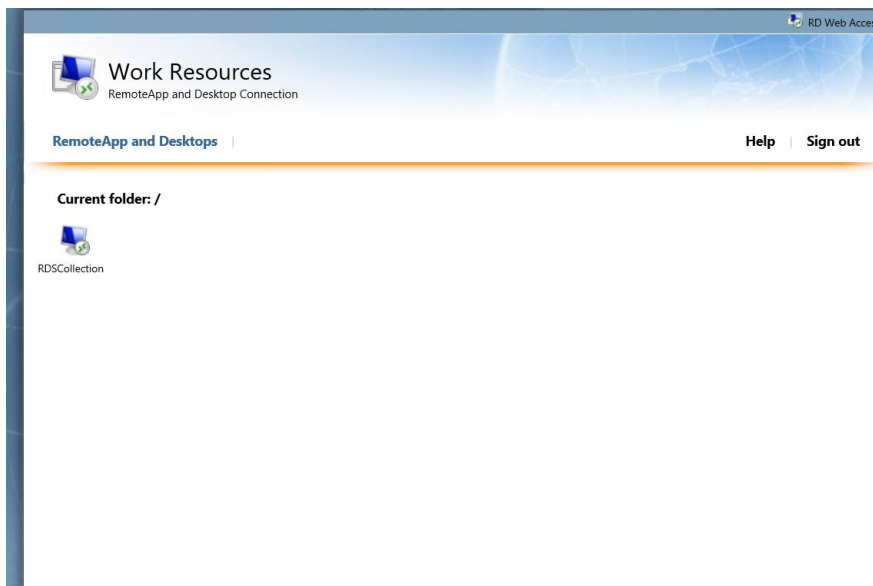
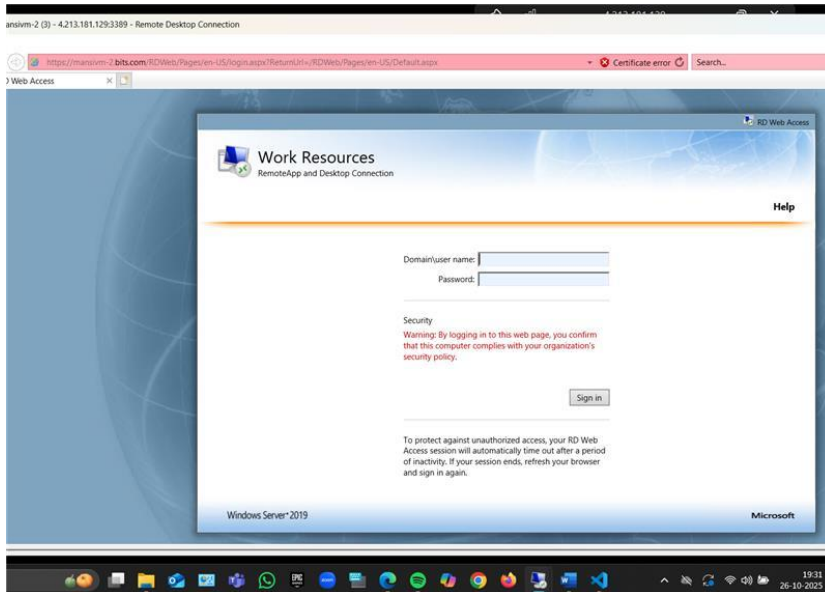


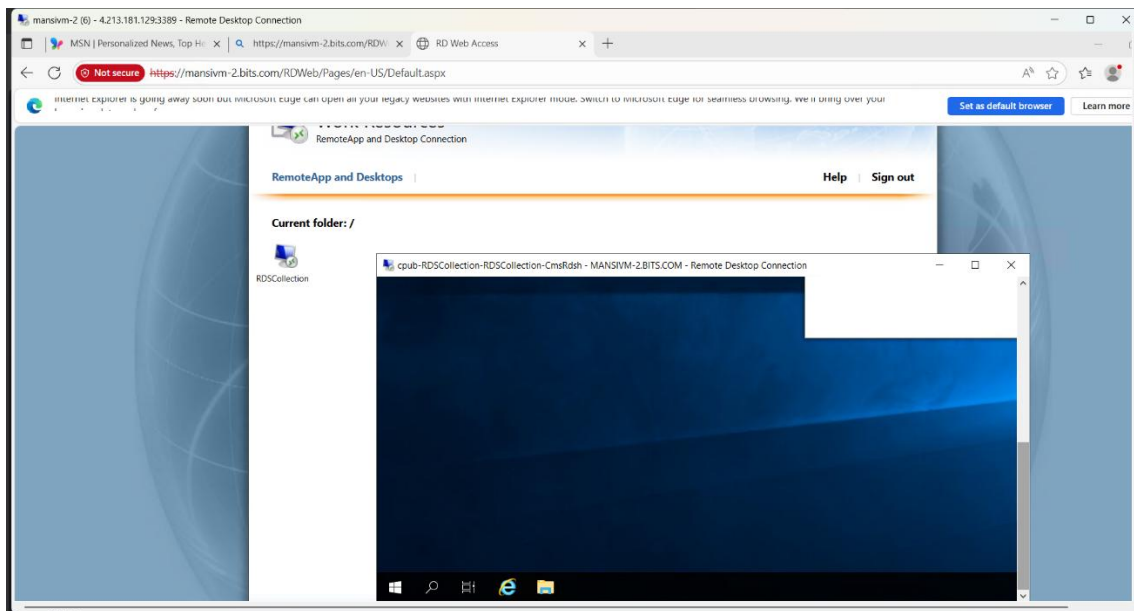
**Validating SSL self-signed certificate:



- **Validating RD Web Access:** <https://mansivm-2.bits.com/RDWeb>

This is the web-based portal where users can log in and launch RemoteApps or full desktops. It's useful for non-technical users who prefer clicking icons instead of using mstsc. In Short, pprovides a browser-based portal where users can launch RemoteApps or full desktops. It's the user-friendly front end.

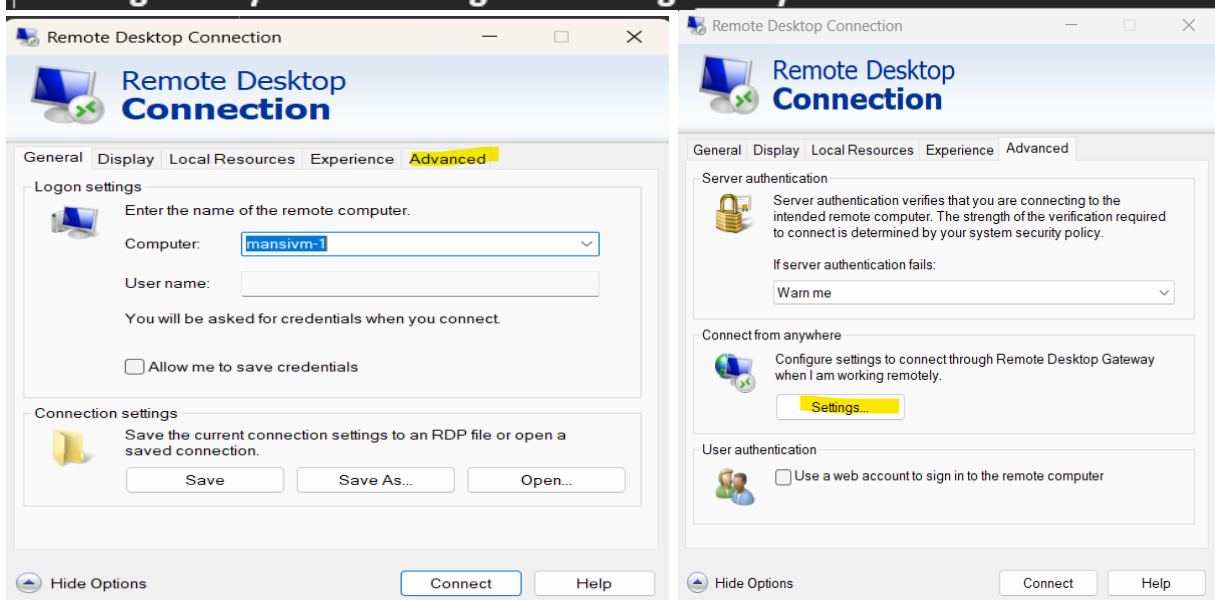


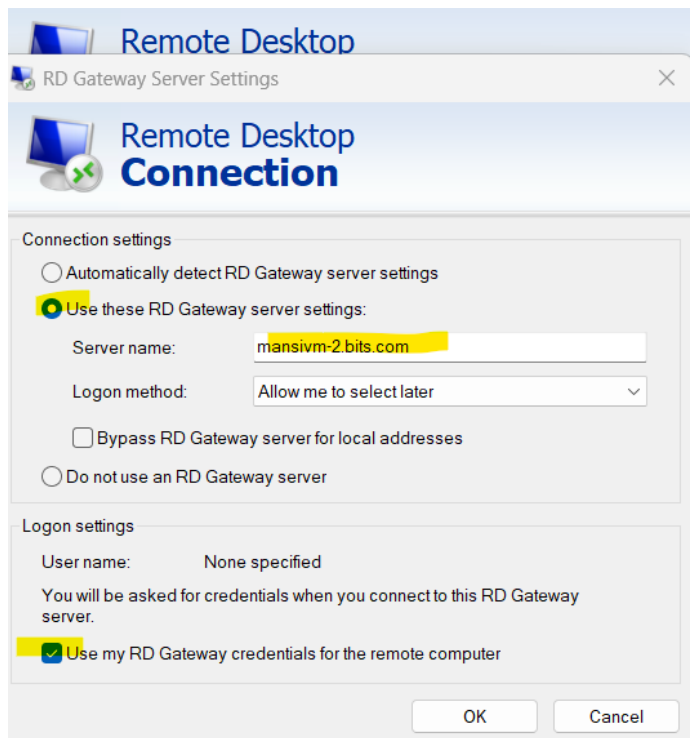


*Testing server's remote connectivity through gateway:

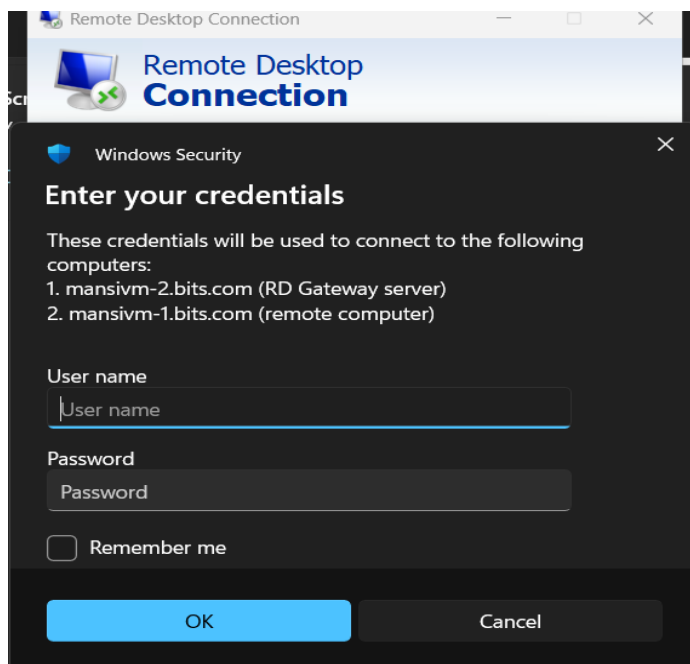
We will try to do a remote desktop to one of our Session host servers. We will add rdsgateway server as Gateway server for it.

Win+r -> type mstsc -> click on advance tab -> Connect from anywhere (settings) -> check use RD gateway server setting and enter gateway server details





Now we will give the required credentials for both gateway server and Session host server



tab

mansivm-0 - Microsoft Azure

New tab

Chat | MANSI VASHISHTH

Microsoft Copilot: Your AI co

Settings

mansivm-2 (6) - 4.213.181.129:3389 - Remote Desktop Connection

D Gateway Manager

ActionViewHelp

RD Gateway Manager

MANSIVM-2 (Local)

Policies

Connection Authorization Policies

Resource Authorization Policies

Monitoring

Monitoring

2 Connection(s) from 1 user(s) to 1 remote computer(s)

Connection ID	User ID	User Name	Connected On	Connection Duration	Idle Time	Target Computer	Client IP Address	Target Port
4:1	BITSMansi	BITSMansi	10/28/2025 11:10:28 AM	00:01:18	00:01:18	mansivm-1.bits.com	fe80::872:1830:78...	3389
3:1	BITSMansi	BITSMansi	10/28/2025 11:10:16 AM	00:01:30	00:00:11	mansivm-1.bits.com	fe80::872:1830:78...	3389

No connection is selected. To view connection details, you must select a connection.

Actions

Monitoring

Select All

Edit Connection Limit

Set Automatic Refresh Options

View

Refresh

Help

Conclusion

- First, I took an Azure subscription and prepared the cloud environment.
- I analysed the requirements and wrote the Terraform code for infrastructure.
- Using terraform apply, I created 3 virtual machines (VMs) in Azure.
- I configured the first VM as a Domain Controller by installing AD DS and DNS.
- I joined the 2nd and 3rd VMs to the created domain.
- On the second VM, I installed SQL Server using PowerShell script.
- The third VM was successfully joined to the domain and kept for extra roles (like SCCM)
- Installed and configured SCCM on mansivm-1, with SQL and all prerequisites handled on the same server.
- SCCM will sync with Microsoft update website with the help of Webserver, download the updates, store them in SQL server and push them to all the client servers.
- Deployed RDS roles on mansivm-2, with mansivm-0 and mansivm-1 as Session Hosts for remote desktop access.
- Used PowerShell to automate the entire RDS setup, including role installation, farm deployment, certificate binding, session collection creation, load balancing, and RD Gateway configuration.
- We are using Azure in setting up this infra which is very useful in cost optimization, efficiency, availability, backup and security.

Detailed Plan Of The Work

S.no:	Tasks to be done	Duration	Plan deliverable
1	Requirement Gathering and analysis	Week 1	Gather all the requirements and make a blueprint of the project
2	Infrastructure Setup with Terraform and azure subscription	Week 2,3 & 4	Started with creating an active Azure subscription and Wrote Terraform code to automatically create the required resources.
3	Virtual Machines Creation	Week 5	Deployed 3 Windows Server 2019 VMs in Azure using Terraform.
4	Create Domain Controller Setup	Week 6	Promote one of the VM as a Domain Controller (AD DS + DNS)
5	Domain Join	Week 7	Join other VMs to domain.
6	Create a Database to store organization data	Week 8 &9	Installed and configured SQL Server 2019 on one of the VM using PowerShell scripts
7	Make device compliance	Week 10 to 11	Deploy an SCCM server and install required updates and applications to make the devices compliance
8	Will set up a Remote Desktop connection broker server	Week 12 to 13	Remote desktop connection broker server will be set up.
9	Will create a Remote Desktop Gateway server and its required roles and Configure and Deploy RDS Farm for DS Domain in Azure environment.	Week 14 to 15	Remote Desktop Gateway server will be created and RDS Farm will be configured and deployed.
10	Testing and validation of the server and setup security	Week 16	Testing and validating the virtual machine and setup security to the machine to protect the VM from unauthorized access

Reference Links:

1. <https://learn.microsoft.com/en-in/training/paths/microsoft-azure-fundamentals-describe-cloud-concepts/>
2. <https://wipro.udemy.com/course/az900-azure/>
3. <https://wipro.udemy.com/course/windows-server-2019-administration-h/>

Check list of items for report:

- Is the Cover page in proper format? Y / N
 - Is the Title page in proper format? Y / N
 - Is the Certificate from the Supervisor in proper format? Has it been signed? Y / N
 - Is Abstract included in the Report? Is it properly written? Y / N
 - Does the Table of Contents' page include chapter page numbers? Y / N
 - Is Introduction included in the report? Is it properly written? Y/N
 - the Pages numbered properly? Y / N
 - Are the Figures numbered properly? Y / N
 - Are the Tables numbered properly? Y / N
 - Are the Captions for the Figures and Tables proper? Y / N
 - Are the Appendices numbered? Y / N
 - Does the Report have Conclusions/ Recommendations of the work? Y / N
 - Are References/ Bibliography given in the Report? Y / N
 - Have the References been cited in the Report? Y / N
- Is the citation of References/ Bibliography in proper format? Y / N